Universidad El Bosque

Facultad Ingeniería

Programa de Ingeniería de Sistemas

Avance 2 - Capa Documental (MongoDB) y Fundamentos Teóricos

Altamira Cine

4 Integrantes:
Jose Espin

Erika Gómez

Estefania Paredes

Santiago Rojas

Presentado a:

Francisco Rafael Barguil Vanegas

Bogotá, Colombia

Noviembre 12 de 2026

Contenido

Repositorio de Git 2
Identidad de la empresa: Altamira Cine 3
Núcleo transaccional en PostgreSQL (Avance 1 corregido) 4
Correcciones incorporadas 4
Modelo entidad-relación (MER) 5
Modelo Relacional 7
Justificación de la normalización 8
Avance 2: persistencia políglota con MongoDB 9
¿Por qué un segundo motor? 9
Arquitectura de la solución 9
Diseño de las cuatro colecciones 11
Validación de esquema 15
Scripts de MongoDB y capturas de ejecución 16
Creación de las colecciones, carga e índices 16
Consultas 34
Validación: inserciones rechazadas 49
Justificación teórica: ACID, BASE y CAP 51
ACID en el núcleo transaccional 51
BASE en los datos documentales 51
Teorema CAP 53
Conclusión 54
Referencias 54

Repositorio de Git

El repositorio del proyecto, con los scripts de PostgreSQL (Avance 1, corregido) y de MongoDB (Avance 2), se encuentra disponible en el siguiente enlace*: aquí*.

Identidad de la empresa: Altamira Cine



Fig. 1. Logo de Altamira Cine.

Altamira Cine es una empresa ubicada en Bogotá, Colombia, cuyo nombre está inspirado en la cueva de Altamira, como representación de la relación entre las imágenes y la narración de historias.

Cuenta con un complejo de seis salas, cinco activas y una en mantenimiento, con una capacidad total de 780 sillas. Su programación incluye estrenos comerciales, cine colombiano y latinoamericano, además de eventos especiales con actores, directores y artistas invitados.

La venta de boletas se realiza presencialmente en taquilla y mediante una plataforma en línea. Se manejan diferentes tarifas según el tipo de espectador y sala, un programa de fidelización por puntos y restricciones de edad según la clasificación de cada película.

El sistema debe controlar la disponibilidad de sillas, impedir ventas para funciones iniciadas o canceladas y gestionar los contratos de los artistas que participan en eventos especiales.

Contexto del Avance 2: Además de las ventas, Altamira Cine genera información que no encaja bien en un modelo de filas fijas: las fichas técnicas de las películas (reparto, versiones y premios), los requisitos técnicos de proyección de cada función, las reseñas de los espectadores y los reportes de incidentes. Esa información tiene estructura variable, crece de forma continua y no exige las garantías de una venta, por lo que se almacena en MongoDB (sección «Avance 2: persistencia políglota con MongoDB»).

Núcleo transaccional en PostgreSQL (Avance 1 corregido)

Esta sección conserva el contenido del Avance 1 e incorpora las correcciones indicadas por el profesor en la retroalimentación. Como el Avance 2 depende del núcleo transaccional, se entrega aquí la versión final y coherente del modelo, los scripts y las evidencias.

Correcciones incorporadas

N.º	**Observación del profesor**	**Corrección aplicada**
1	La evidencia del script 02 mostraba solo el conteo por UNION ALL, sin los INSERT.	Se capturó la ejecución de los INSERT (salidas INSERT 0 n) y, posteriormente, el conteo por tabla.
2	El modelo relacional no coincidía con el script: PK compuesta en contratos, atributo duplicado y FK con nombres distintos (SALAS_id_sala, STAFF_id_staff).	Se rehízo el modelo con id_contrato como única PK, sin atributos duplicados y con las FK nombradas como en el DDL (id_sala, id_staff, etc.). Los tipos de datos corresponden a PostgreSQL.
3	Dependencia transitiva en funciones: titulo_pelicula determina género, clasificación y duracion_min, pero se declaraba 3FN.	Se creó la entidad peliculas (id_pelicula, titulo, genero, clasificacion, duracion_min). La tabla funciones solo guarda id_pelicula, idioma y horario. Ahora son 9 tablas y el modelo cumple 3FN. Los triggers de programación y edad, así como el reporte de ventas, consultan la información de peliculas.
4	Capturas sin la sentencia que produce el resultado (CALL tapado, editor mostrando otra consulta y falta de la línea TOTAL).	Cada procedimiento y consulta se ejecuta por separado. Las capturas muestran la sentencia y su resultado, con las notificaciones visibles hasta la línea TOTAL.
5	Presentación: figuras sin citar, modelo entidad-relación ilegible, cifras erróneas (nueve FK frente a ocho creadas) y estilo de código desigual.	Todas las figuras y tablas se citan por número. El MER y el modelo relacional se presentan en página horizontal y en formato vectorial. Las cifras se verificaron contra el código (9 tablas, 9 FK, 15 índices, 8 triggers y 24 pruebas). El código utiliza una sola fuente monoespaciada, incluidos los CALL.
6	La ciudad solo aparecía en la portada.	Se agregó la ciudad en la sección de identidad, indicando que Altamira Cine opera en Bogotá, Colombia.

Tabla 1. Correcciones del Avance 1.

Modelo entidad-relación (MER)

El modelo está compuesto por nueve entidades: Salas, Tipos_Boleta, Actores_Directores, Staff, Peliculas, Espectadores, Funciones, Contratos y Ventas. Las ocho primeras definidas en el enunciado se complementan con Peliculas, que separa la ficha básica de una película de cada una de sus proyecciones.

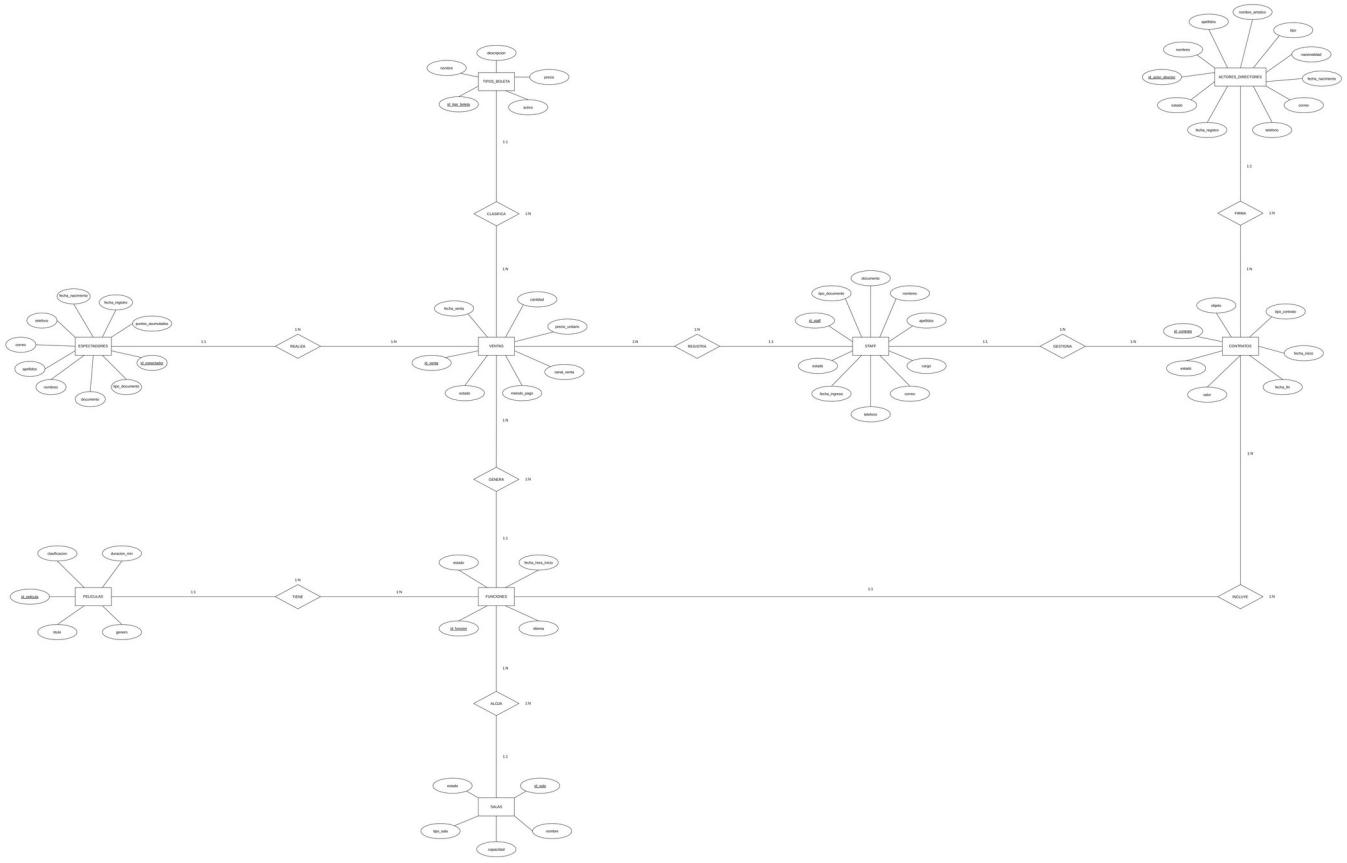


Fig. 2. Modelo entidad-relación de Altamira Cine

Relación	Entidades	Cardinalidad	Explicación
ALOJA	Salas — Funciones	1:N	Una sala aloja muchas funciones; cada función se proyecta en una sola sala.
PROGRAMA	Películas — Funciones	1:N	Una película se programa en muchas funciones; cada función proyecta una sola película. Una película puede existir sin funciones.
GENERA	Funciones — Ventas	1:N	Una función genera muchas ventas; cada venta es para una sola función.
REALIZA	Espectadores — Ventas	1:N	Un espectador realiza muchas compras; cada venta pertenece a un solo espectador.
CLASIFICA	Tipos_Boleta — Ventas	1:N	Un tipo de boleta (general, estudiante, VIP, etc.) clasifica muchas ventas.
REGISTRA	Staff — Ventas	1:N (opcional)	Un taquillero registra muchas ventas. En el canal Online no interviene ningún empleado, por eso la participación es opcional del lado de la venta.
GESTIONA	Staff — Contratos	1:N	Un administrador o coordinador gestiona muchos contratos; cada contrato lo gestiona un solo empleado.
FIRMA	Actores_Directores — Contratos	1:N	Un artista firma muchos contratos a lo largo del tiempo; cada contrato es con un solo artista.
INCLUYE	Funciones — Contratos	1:N	Una función especial puede incluir varios contratos (varios artistas); cada contrato corresponde a una sola función.

Tabla 2. Relaciones entre las entidades, cardinalidades y reglas de negocio que representan.

Contratos y Ventas concentran las transacciones del negocio. Las relaciones de muchos a muchos propias del negocio, entre artistas y funciones y entre espectadores y funciones, se resuelven mediante estas dos entidades, que además guardan atributos propios de cada relación (valor y vigencia del contrato; cantidad y precio de la venta). Por eso no se necesitaron tablas intermedias adicionales.

Modelo Relacional

El modelo relacional final (Fig. 3) quedó alineado con el script de PostgreSQL: cada tabla tiene una única llave primaria id_* autogenerada, las llaves foráneas se llaman igual que la columna que referencian (id_sala, id_staff, …) y los tipos son los de PostgreSQL. En contratos la llave primaria es solo id_contrato; la combinación artista-función se protege con una restricción UNIQUE. En ventas, id_staff admite NULL (venta online). Hay valores que se calculan en lugar de almacenarse: el total de una venta (cantidad × precio unitario) y la hora de fin de una función (inicio + duración de la película). Así no quedan datos derivados que puedan desactualizarse. En cambio, precio_unitario sí se guarda en la venta para conservar el valor histórico aunque la tarifa cambie.

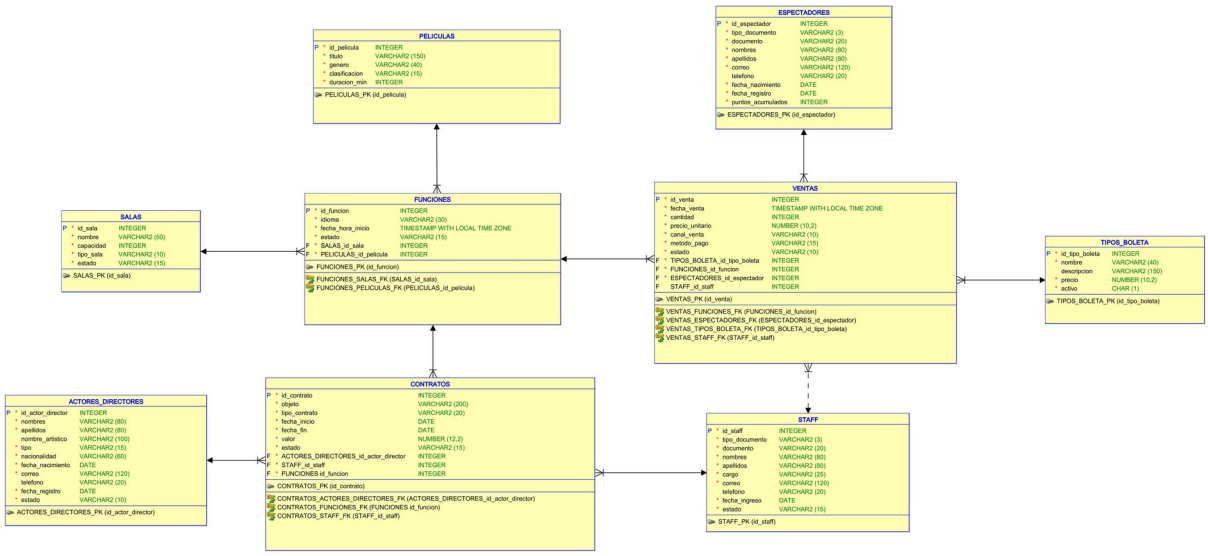


Fig. 3. Modelo relacional de Altamira Cine.

Justificación de la normalización

El modelo se verificó contra las tres primeras formas normales. Todas las tablas usan como llave primaria un identificador autogenerado y los demás datos que identifican a una persona u objeto del negocio (documento, correo, nombre de sala, título de película) se protegen con restricciones UNIQUE.

Primera forma normal (1FN): Todos los atributos son atómicos y no hay grupos repetidos dentro de una fila: nombres y apellidos están en columnas separadas, cada fila de ventas corresponde a un solo tipo de boleta y cada contrato a un solo artista y una sola función. Lo que puede ser múltiple (boletas de una función, contratos de un artista) se modeló como filas de otra tabla.

Segunda forma normal (2FN): Todas las llaves primarias tienen una sola columna, de modo que ningún atributo puede depender de solo una parte de la llave. En contratos, la llave es únicamente id_contrato.

Tercera forma normal (3FN): Ningún atributo no clave depende de otro atributo no clave. La capacidad no se repite en funciones (depende de la sala); los datos del espectador no se copian en ventas (solo id_espectador); el cargo del empleado no se copia en contratos (se consulta en staff y lo valida un trigger). Y, tras la corrección del Avance 1, el título, el género, la clasificación y la duración ya no están en funciones: viven en peliculas, de modo que cambiar la clasificación de una película es una sola actualización y no pueden quedar funciones con valores contradictorios. El modelo cumple 3FN sin excepciones.

Avance 2: persistencia políglota con MongoDB

¿Por qué un segundo motor?

El modelo relacional del Avance 1 resuelve muy bien lo transaccional: vender una boleta sin superar el aforo, anular una venta junto con sus puntos o cancelar una función con todo lo que depende de ella. Pero Altamira Cine produce otra información con características distintas: estructura variable (el reparto y los premios cambian de una película a otra; una función 3D, IMAX o VIP exige requisitos distintos de una 2D; cada tipo de incidente trae campos propios), crecimiento continuo (reseñas e incidentes) y menor exigencia de consistencia inmediata. Forzar esos datos en tablas obligaría a columnas opcionales y *joins* costosos; llevar las ventas a un motor documental sacrificaría llaves foráneas y transacciones. La persistencia políglota consiste en usar cada motor donde es más fuerte [1], [2]. La Tabla 3 resume el reparto.

Dato	Motor	Razón principal
Salas, funciones, películas (datos básicos), espectadores, staff, tipos de boleta	PostgreSQL	Datos maestros con relaciones y restricciones; fuente de verdad.
Ventas, contratos, puntos de fidelización	PostgreSQL	Invariantes entre filas (cupo, edad, reembolsos) que exigen transacciones ACID.
Ficha técnica de la película	MongoDB	Estructura variable; se lee completa en un solo documento.
Requisitos técnicos de proyección por función	MongoDB	Campos distintos según el formato de sala; equipos y checklist anidados.
Reseñas de los espectadores	MongoDB	Crecimiento ilimitado, campos opcionales y consistencia eventual aceptable.
Reportes de incidentes	MongoDB	Tres tipos con detalle distinto; las acciones de seguimiento se agregan con el tiempo.

Tabla 3. Reparto de los datos entre PostgreSQL y MongoDB.

Arquitectura de la solución

La Fig. 4 muestra la solución: PostgreSQL es el núcleo transaccional y MongoDB la capa documental.

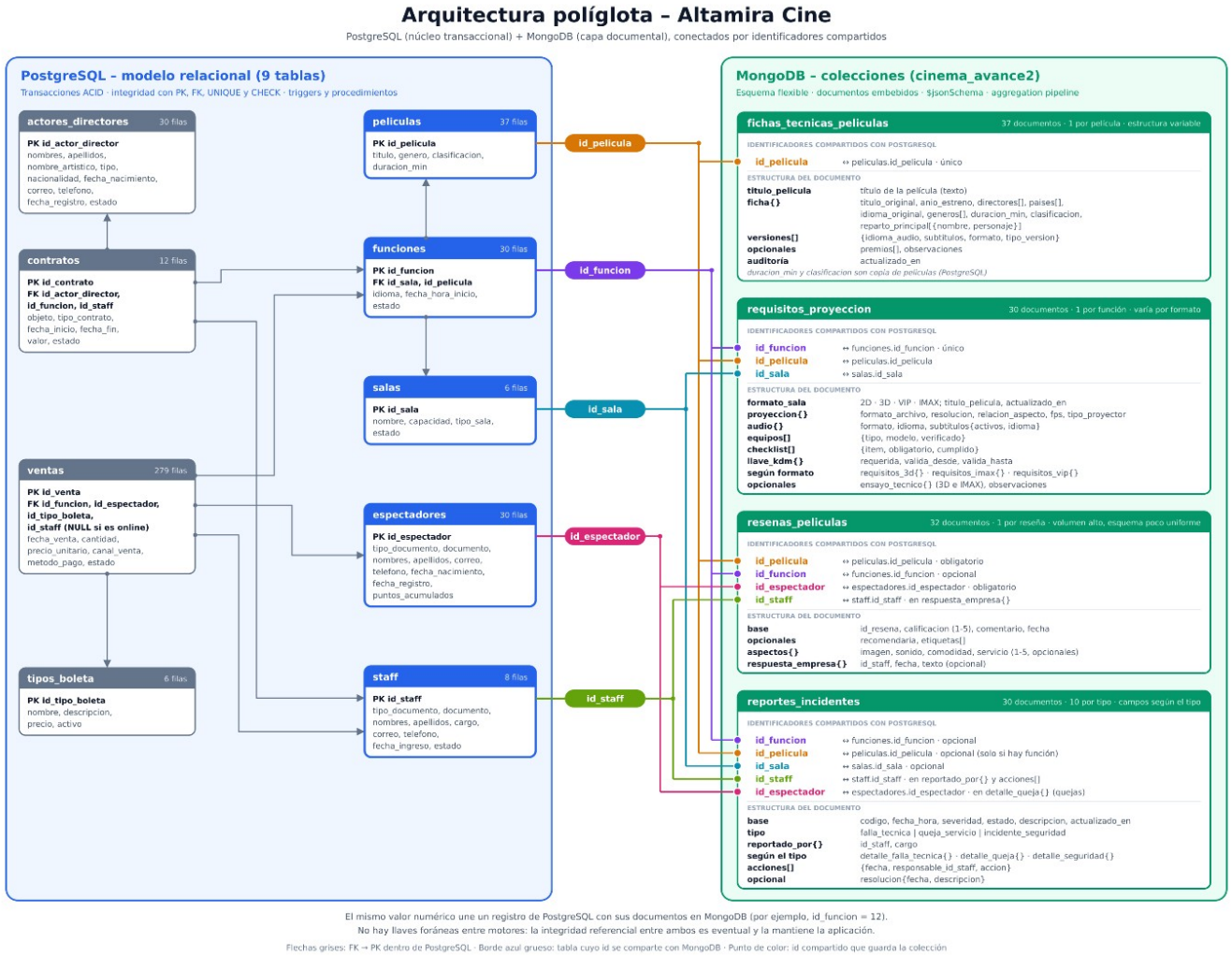


Fig. 4. Arquitectura políglota de Altamira Cine: PostgreSQL para lo transaccional y MongoDB para lo documental.

Diseño de las cuatro colecciones

Las colecciones son *fichas_tecnicas_peliculas*, *requisitos_proyeccion*, *resenas_peliculas* y *reportes_incidentes*. En MongoDB la decisión central de diseño es qué se *embebe* dentro del documento y qué se *referencia* por identificador [3]. Se aplicaron los criterios de la Tabla 4, y la Tabla 5 resume lo decidido en cada colección.

Se embebe cuando…	Se referencia cuando…
La relación es 1:1 o 1:pocos y el arreglo tiene un máximo claro.	La relación es 1:muchos sin límite (el documento podría crecer sin control y acercarse al límite de 16 MB).
Los datos se leen casi siempre juntos con el documento padre.	Se consultan de forma independiente.
Se actualizan juntos y requieren atomicidad: una escritura sobre un solo documento es atómica.	Los comparten muchos documentos o cambian por su cuenta.
El dato hijo no tiene sentido sin el padre.	Tienen ciclo de vida propio, volumen alto o caducidad distinta.

Tabla 4. Criterios para decidir entre documento embebido y referenciado.

La Tabla 4 establece el criterio general que se aplicó para decidir si un dato se embebe dentro del documento o se referencia por identificador. Un dato se embebe cuando la relación con el documento padre es de uno a uno o de uno a pocos y el arreglo tiene un máximo claramente acotado; cuando se lee casi siempre junto con dicho documento; cuando se actualiza junto con él, de modo que una escritura sobre un solo documento sea atómica; y cuando el dato hijo carece de sentido sin el padre. Por el contrario, un dato se referencia cuando la relación es de uno a muchos sin límite, lo que haría crecer el documento sin control y acercarlo al límite de 16 MB; cuando se consulta de forma independiente; cuando lo comparten muchos documentos o cambia por su cuenta; o cuando tiene ciclo de vida propio, volumen alto o una caducidad distinta.

Colección	Embebido	Referenciado	Justificación
`fichas_tecnicas_peliculas` (37 docs)	`ficha{}` (directores[], paises[], generos[], reparto_principal[]), `versiones[]`, `premios[]`, `observaciones`	`id_pelicula`	Una ficha por película, no por función. Reparto, versiones y premios son acotados y se leen siempre con la película.
`requisitos_proyeccion` (30 docs)	`proyeccion{}`, `audio{}`, `equipos[]`, `checklist[]`, `llave_kdm{}`, `requisitos_3d{}`, `requisitos_imax{}`, `requisitos_vip{}`, `ensayo_tecnico{}`	`id_funcion`, `id_pelicula`, `id_sala`	Un documento por función. El equipo de proyección lo lee completo antes de la función.

Colección	Embebido	Referenciado	Justificación
`resenas_peliculas` (32 docs)	`aspectos{}`, `respuesta_empresa{}`, `etiquetas[]`	`id_pelicula`, `id_funcion` (opcional), `id_espectador`; `id_staff` en `respuesta_empresa`	Las reseñas crecen sin límite: no se embeben en la ficha. Aspectos y respuesta solo se leen con la reseña.
`reportes_incidentes` (30 docs)	`reportado_por{}`, `detalle_falla_tecnica{}` / `detalle_queja{}` / `detalle_seguridad{}`, `acciones[]`, `resolucion{}`	`id_funcion`, `id_pelicula`, `id_sala` (opcionales); `id_staff`; `id_espectador` (quejas)	Cada tipo lleva su propio detalle. Las acciones se agregan al mismo documento y su escritura es atómica.

Tabla 5. Decisiones de embebido y referencia por colección.

La Tabla 5 aplica estos criterios a cada colección. En fichas_tecnicas_peliculas se embeben la ficha, las versiones, los premios y las observaciones, y se referencia id_pelicula, porque el reparto, las versiones y los premios son listas acotadas que se leen siempre con la película; la ficha se diseña por película y no por función. En requisitos_proyeccion se embebe el conjunto técnico de cada función (proyección, audio, equipos, checklist, llave KDM y los requisitos según el formato), y se referencian id_funcion, id_pelicula e id_sala, ya que cada documento corresponde a una función. En resenas_peliculas se embeben los aspectos valorados y la respuesta de la empresa, y se referencian la película, la función y el espectador, dado que las reseñas crecen sin límite y no deben incorporarse a la ficha. Por último, en reportes_incidentes se embeben el responsable del reporte, el detalle propio de cada tipo de incidente, las acciones y la resolución, pues el seguimiento de un incidente se mantiene en un mismo documento; las acciones[] se embeben porque son pocas por incidente y su número está acotado, así que no encajan en el criterio de 1:muchos sin límite; se referencian la función, la película, la sala, el personal y, cuando aplica, el espectador.

Colección 1: fichas_tecnicas_peliculas

Un documento por película: 37, de los cuales los ids 1 a 25 tienen funciones programadas y los ids 26 a 37 son cine colombiano y latinoamericano aún sin programar.

- **Se embebe** la ficha (directores, países, géneros y reparto), las versiones y los premios porque son listas cortas y acotadas, se leen siempre junto con la película y no tienen sentido sin ella. Las versiones (idioma de audio, subtítulos y formato) son lo que permite decidir cuántas copias necesita cada película.
- **Se referencia** `id_pelicula`, que enlaza con `peliculas` en PostgreSQL y es único en la colección. La ficha es por película y no por función, de modo que las películas con dos funciones (Coco, Encanto, Barbie, Parásitos, Mad Max) tienen una sola ficha.
- **Copia controlada:** `duracion_min` y `clasificacion` repiten el valor de PostgreSQL para no consultar dos motores al mostrar la ficha.
- **Opcionales:** `premios` y `observaciones`, porque no todas las películas los tienen.

Colección 2: requisitos_proyeccion

Un documento por función: 30, con el `id_pelicula` y el `id_sala` reales de cada una (14 funciones 2D, 5 VIP, 5 3D y 6 IMAX).

- **Se embebe** todo lo técnico (proyección, audio, equipos, checklist y llave KDM) porque lo lee completo el proyeccionista antes de cada función y no existe sin ella. Los objetos `requisitos_3d`, `requisitos_imax` y `requisitos_vip` aparecen solo en el formato que los necesita, y `ensayo_tecnico` solo en 3D e IMAX: ese es el esquema variable.
- **Se referencia** `id_funcion` (único), `id_pelicula` e `id_sala`, que viven en PostgreSQL. No se copia nada de la película ni de la sala salvo el título.

Colección 3: resenas_peliculas

Un documento por reseña: 32, cada una de un espectador con boleta pagada para una función ya finalizada (ids 1 a 16, donde `id_pelicula` coincide con `id_funcion`).

- **Se referencia** `id_pelicula`, `id_funcion` (opcional) e `id_espectador`. Las reseñas **no se embeben en la ficha** porque crecen sin límite (riesgo del límite de 16 MB) y se consultan por película y por espectador de forma independiente.
- **Se embebe** `aspectos` (imagen, sonido, comodidad, servicio: relación 1:1) y `respuesta_empresa` (a lo sumo una respuesta del staff): solo se leen junto con la reseña. Guardan `id_staff` como referencia, no el nombre del empleado.
- **Opcionales:** `recomendaria`, `etiquetas`, `aspectos` y `respuesta_empresa`, por eso el esquema es poco uniforme.

Colección 4: reportes_incidentes

Un documento por incidente: 30, diez de cada tipo (`falla_tecnica`, `queja_servicio`, `incidente_seguridad`). Los campos comunes están en la raíz y cada tipo trae su propio objeto de detalle (patrón polimórfico); el `$jsonSchema` obliga a que venga el que corresponde.

- **Se embebe** `reportado_por`, el detalle según el tipo, `acciones[]` y `resolucion`: el seguimiento de un incidente se agrega al mismo documento, se lee junto con él y se actualiza de forma atómica.
- **Se referencia** `id_funcion`, `id_pelicula` e `id_sala` (opcionales: hay incidentes en el hall, el parqueadero o la sala 4DX en mantenimiento, sin función), `id_staff` (quien reporta y los responsables de cada acción) e `id_espectador` (solo en quejas).

Los otros dos tipos reemplazan el detalle de falla técnica por su propio objeto: una queja (INC-012) lleva detalle de queja, con el espectador, la categoría, el canal y la compensación, y un incidente de seguridad (INC-023) lleva detalle de seguridad, con el tipo de evento, la zona, las personas involucradas y la autoridad llamada.

Validación de esquema

Las cuatro colecciones se crean con `$jsonSchema`, `validationLevel: "strict"` y `validationAction: "error"` [4].

Colección	Campos obligatorios	Reglas destacadas
`fichas_tecnicas_peliculas`	id_pelicula, titulo_pelicula, ficha, versiones, actualizado_en	anio_estreno entre 1888 y 2100; duracion_min entre 1 y 400; clasificacion en {Todo público, +7, +12, +15, +18}; versiones con al menos un elemento, formato en {2D, 3D, VIP, IMAX, 4DX}
`requisitos_proyeccion`	id_funcion, id_pelicula, id_sala, titulo_pelicula, formato_sala, proyeccion, audio, equipos, checklist, llave_kdm, actualizado_en	resolucion en {2K, 4K}; fps en {24, 48, 60}; equipos y checklist con al menos un elemento; requisitos_3d exige tipo_gafas y gafas_requeridas
`resenas_peliculas`	id_resena, id_pelicula, id_espectador, calificacion, comentario, fecha	calificacion entre 1 y 5; comentario de 5 a 1000 caracteres; aspectos entre 1 y 5; respuesta_empresa exige id_staff, fecha y texto
`reportes_incidentes`	codigo, tipo, fecha_hora, severidad, estado, descripcion, reportado_por, acciones, actualizado_en	codigo con patrón INC-000; severidad en {baja, media, alta, critica}; descripcion de al menos 10 caracteres; un anyOf obliga a que venga el detalle que corresponde al tipo

Tabla 6. Validación $jsonSchema por colección.

La Tabla 6 resume la validación que MongoDB aplica en el momento de la escritura. Cada colección se creó con un esquema $jsonSchema en modo estricto (validationLevel: "strict") y con acción de error (validationAction: "error"), de manera que un documento que no cumpla el esquema se rechaza y no se almacena. En fichas_tecnicas_peliculas, los campos obligatorios son el identificador de la película, el título, la ficha, las versiones y la fecha de actualización; el año de estreno debe estar entre 1888 y 2100, la duración entre 1 y 400 minutos y la clasificación debe ser Todo público, +7, +12, +15 o +18. En requisitos_proyeccion, la resolución solo puede ser 2K o 4K, la velocidad debe ser 24, 48 o 60 fps, y los equipos y el checklist deben tener al menos un elemento; además, los requisitos 3D deben indicar el tipo de gafas y la cantidad requerida. En resenas_peliculas, la calificación debe estar entre 1 y 5, el comentario entre 5 y 1000 caracteres, y toda respuesta de la empresa debe identificar al miembro del personal que la emite, con su fecha y su texto. En reportes_incidentes, el código debe seguir el patrón INC-000, la severidad debe ser baja, media, alta o crítica, y una condición anyOf exige que el documento contenga el detalle correspondiente a su tipo. Estas reglas se comprueban con los intentos de inserción rechazada presentados en las Figuras 12a a 12d.

Scripts de MongoDB y capturas de ejecución

Cada colección tiene tres scripts que se ejecutan con *mongosh* en este orden: 01 (creación con validación e índices), 02 (datos de prueba) y 03 (consultas). Los de creación borran la colección y la vuelven a crear, y los de datos la vacían antes de insertar, así que se pueden repetir. Los identificadores corresponden a los datos de PostgreSQL (`id_pelicula` entre 1 y 37, `id_funcion` entre 1 y 30, `id_sala` entre 1 y 6, `id_espectador` entre 1 y 30). Cada captura muestra la sentencia ejecutada junto con su resultado.

Creación de las colecciones, carga e índices

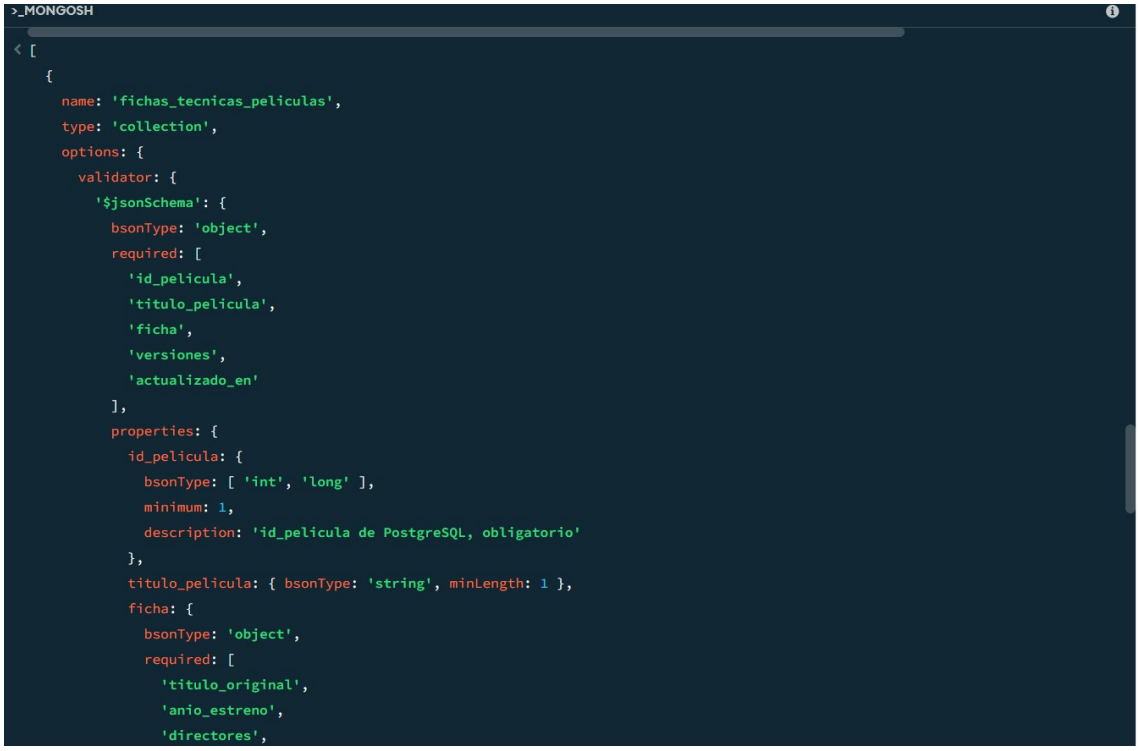


Fig. 5a. Esquema de fichas_tecnicas_peliculas (getCollectionInfos, inicio del validador $jsonSchema).



Fig. 5b. Validador $jsonSchema de fichas_tecnicas_peliculas (propiedades de la ficha).



Fig. 5c. Validador de fichas_tecnicas_peliculas (premios, nivel y acción de validación).

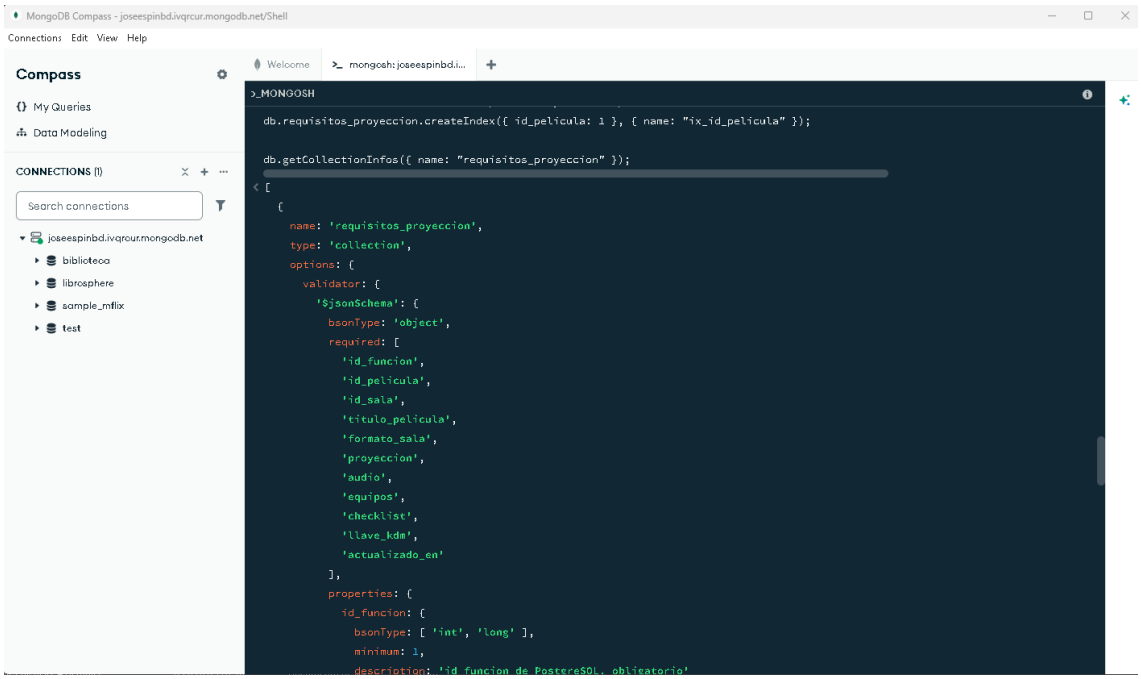


Fig. 5d. Esquema de requisitos_proyeccion (getCollectionInfos, inicio del validador $jsonSchema).

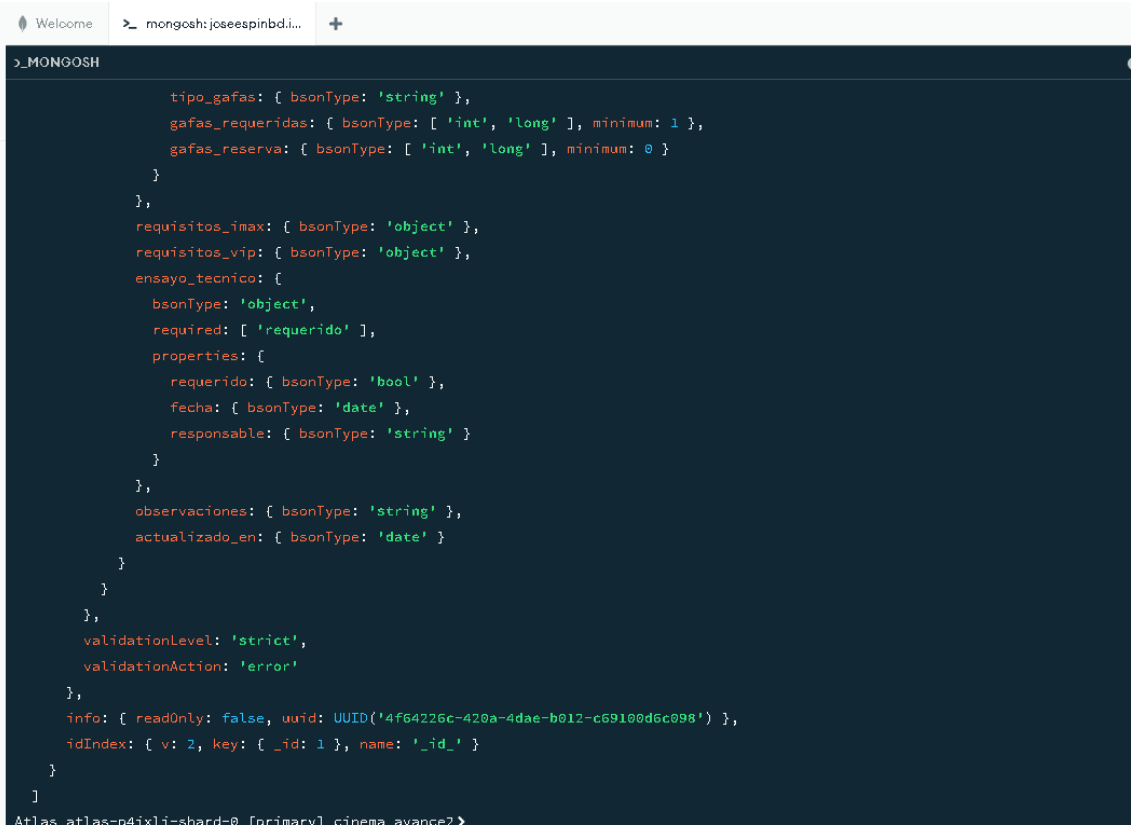


Fig. 5e. Validador de requisitos_proyeccion (final del validador, nivel strict y acción error).

```
          properties: {
            imagen: { bsonType: ["int", "long"], minimum: 1, maximum: 5 },
            sonido: { bsonType: ["int", "long"], minimum: 1, maximum: 5 },
            comodidad: { bsonType: ["int", "long"], minimum: 1, maximum: 5 },
            servicio: { bsonType: ["int", "long"], minimum: 1, maximum: 5 }
          }
        },
        respuesta_empresa: {
          bsonType: "object",
          required: ["id_staff", "fecha", "texto"],
          properties: {
            id_staff: {
              bsonType: ["int", "long"],
              minimum: 1,
              description: "id_staff de PostgreSQL"
            },
            fecha: { bsonType: "date" },
            texto: { bsonType: "string", minLength: 5 }
          }
        }
      }
    }
  },
  validationLevel: "strict",
  validationAction: "error"
});
{ ok: 1 }
```

Fig. 5f. Validador de resenas_peliculas (aspectos y respuesta_empresa).

Welcome | mongosh: dionysus.dfc7...

>_MONGOSH

```
    app.asar.unpacked/node_modules/@mongosh/node-runtime-worker-thread/dist/worker-runtime.js:2:6549696)
    npacked/node_modules/@mongosh/node-runtime-worker-thread/dist/worker-runtime.js:2:6545340
> // Avance 2 - MongoDB - Coleccion reportes_incidentes
  // Responsable: Estefania Paredes (tef333)
  //
  // Crea la coleccion con validacion $jsonSchema y sus indices.
  // Los incidentes tienen campos distintos segun el tipo (falla tecnica,
  // queja de servicio o incidente de seguridad): cada tipo lleva su propio
  // objeto de detalle, y el $jsonSchema obliga a que venga el que corresponde.
  // Identificadores compartidos con PostgreSQL: id_funcion, id_pelicula, id_sala,
  // id_staff (reportado_por, responsables) e id_espectador (quejas).
  // Se puede ejecutar varias veces: borra la coleccion y la vuelve a crear.
  // Orden: 01 -> 02 -> 03

  db.reportes_incidentes.drop();

  db.createCollection("reportes_incidentes", {
    validator: {
      $jsonSchema: {
        bsonType: "object",
        required: ["codigo", "tipo", "fecha_hora", "severidad", "estado", "descripcion", "reportado_por", "acciones", "actu
        properties: {
          codigo: {
            bsonType: "string",
            pattern: "^INC-[0-9]{3,}$",
            description: "codigo del incidente, por ejemplo INC-001"
          },
          tipo: {
            enum: ["falla_tecnica", "queja_servicio", "incidente_seguridad"]
          },
          fecha_hora: {
            bsonType: "date"
          },
          id_funcion: {
            bsonType: ["int", "long"],
            minimum: 1,
            description: "id_funcion de PostgreSQL, opcional (un incidente puede ocurrir fuera de una funcion)"
          },
```

Fig. 5g. Script de creación de reportes_incidentes con validador $jsonSchema.

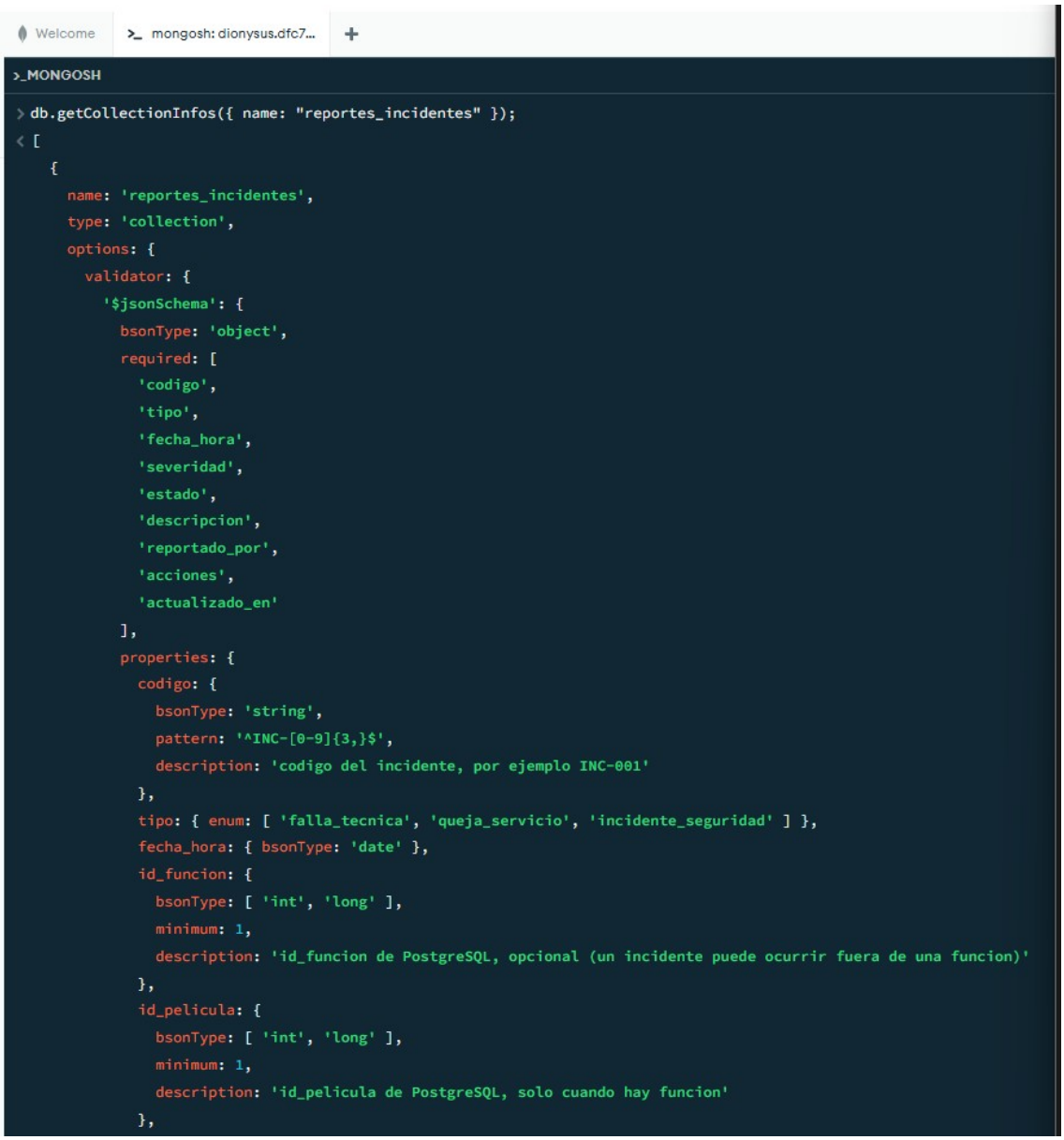


Fig. 5h. Esquema de reportes_incidentes (getCollectionInfos).

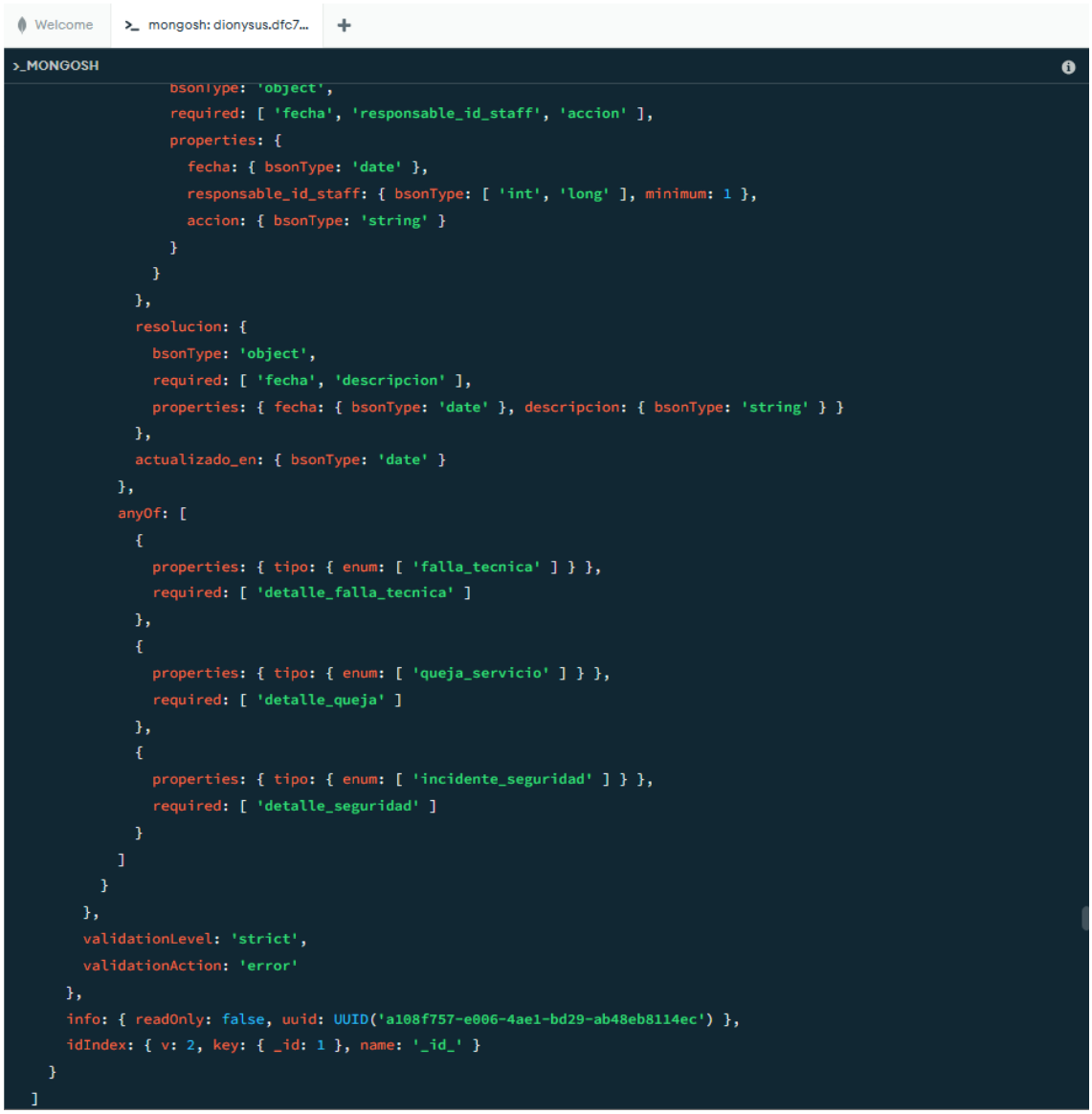


Fig. 5i. Validador de reportes_incidentes (reglas por tipo de incidente).



Fig. 6a. Resultado de insertOne en fichas_tecnicas_peliculas (acknowledged: true).



Fig. 6b. Conteo de documentos de fichas_tecnicas_peliculas (countDocuments).

```
> db.requisitos_proyeccion.insertOne({
    id_funcion: 1,
    id_pelicula: 1,
    id_sala: 1,
    titulo_pelicula: "Oppenheimer",
    formato_sala: "2D",
    proyeccion: {
      formato_archivo: "DCP",
      resolucion: "2K",
      relacion_aspecto: "2.39:1",
      fps: 24,
      tipo_proyector: "Xenón",
      brillo_objetivo_fl: 14
    },
    audio: {
      formato: "7.1",
      idioma: "Inglés",
      subtitulos: {
        activos: true,
        idioma: "Español"
      }
    },
    equipos: [
```

Fig. 6c. Inserción de requisitos_proyeccion (insertOne, función 1).

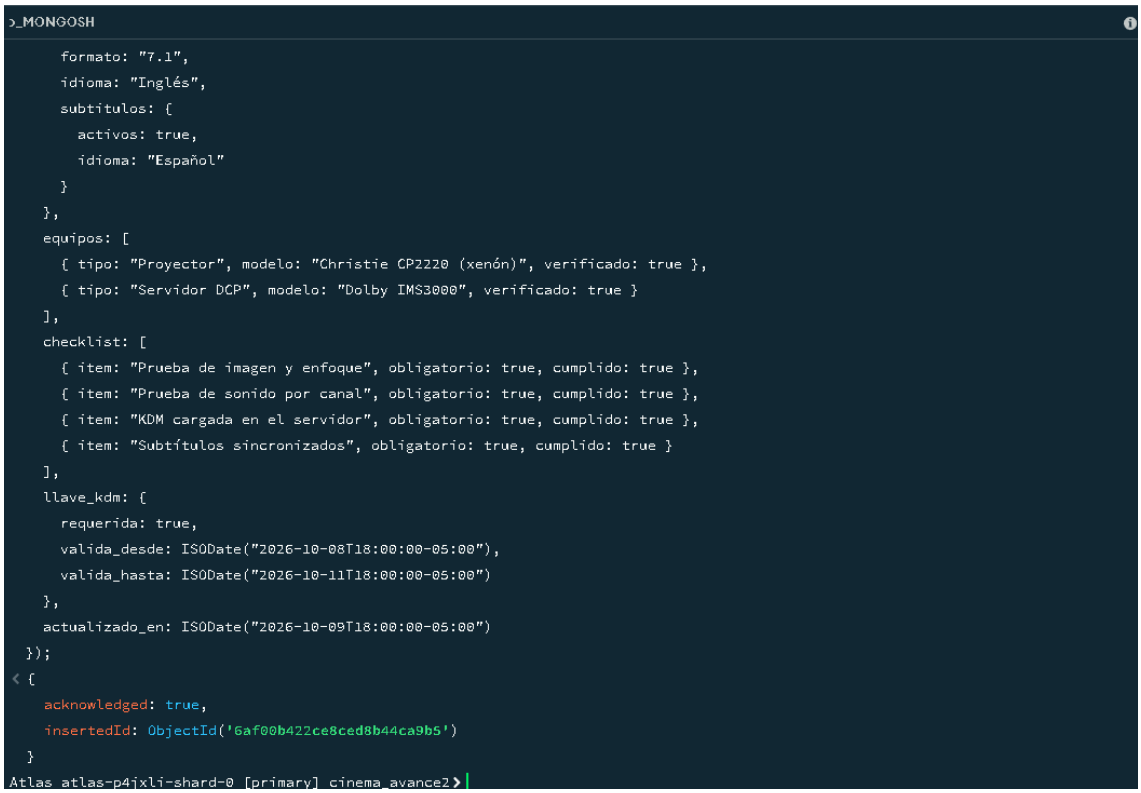


Fig. 6d. Resultado de insertOne en requisitos_proyeccion (acknowledged: true).

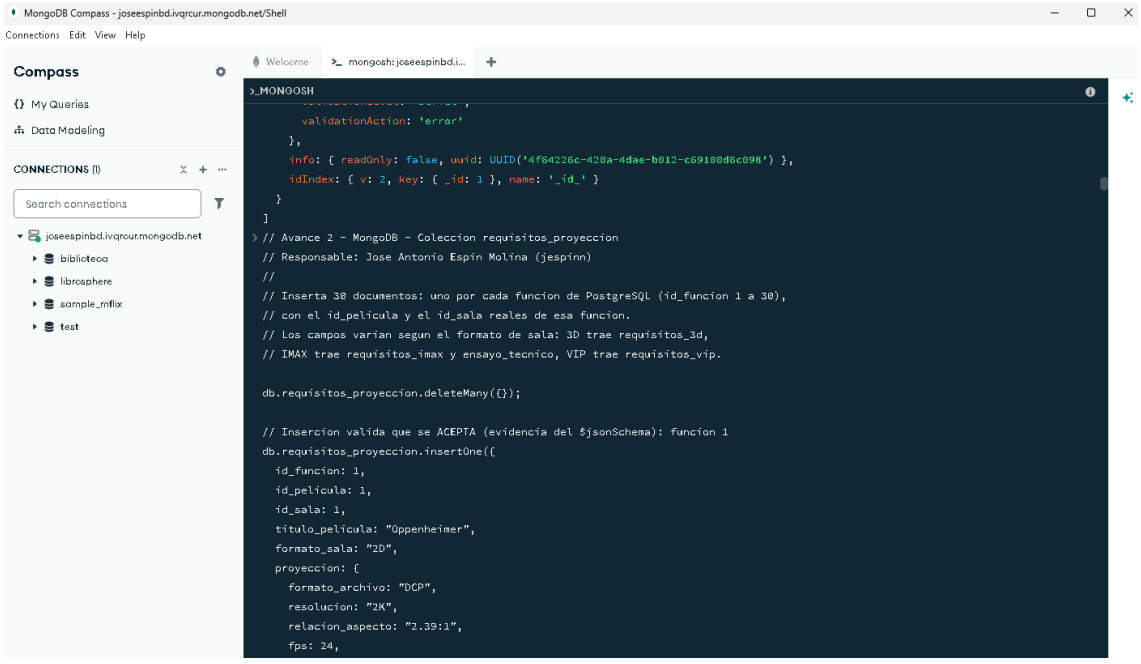


Fig. 6e. Script de carga de requisitos_proyeccion con insertOne válido.

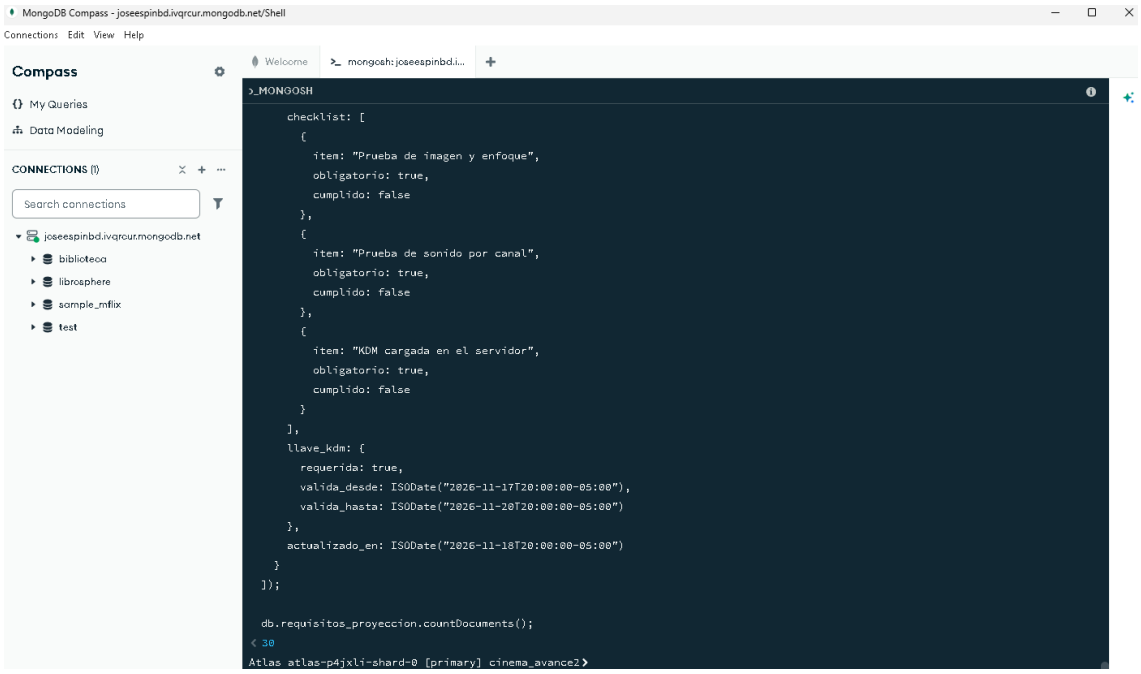


Fig. 6f. Checklist y conteo de requisitos_proyeccion (30 documentos).

```
db.resenas_peliculas.insertOne(
{
  id_resena: 1,
  id_pelicula: 1,
  id_funcion: 1,
  id_espectador: 4,
  calificacion: 5,
  comentario: "Actuación impecable y un guion muy bien armado, aunque se siente larga.",
  fecha: ISODate("2026-10-10T12:00:00-05:00"),
  recomendaria: true,
  etiquetas: [
    "actuaciones",
    "guion"
  ],
  aspectos: {
    imagen: 5,
    sonido: 4
  }
});

// Resto de documentos: resenas 2 a 32
db.resenas_peliculas.insertMany([
  {
    id_resena: 2,
    id_pelicula: 1,
    id_funcion: 1,
```

Fig. 6g. Inserción de una reseña en resenas_peliculas (insertOne).

```
> db.resenas_peliculas.insertOne(
  {
    id_resena: 1,
    id_pelicula: 1,
    id_funcion: 1,
    id_espectador: 4,
    calificacion: 5,
    comentario: "Actuación impecable y un guion muy bien armado, aunque se siente larga.",
    fecha: ISODate("2026-10-10T12:00:00-05:00"),
    recomendaria: true,
    etiquetas: [
      "actuaciones",
      "guion"
    ],
    aspectos: {
      imagen: 5,
      sonido: 4
    }
  });
< {
    acknowledged: true,
    insertedId: ObjectId('6af577e303e15cf61692d69e')
  }
ProyectoBD>
```

Fig. 6h. Resultado de insertOne en resenas_peliculas (acknowledged: true).

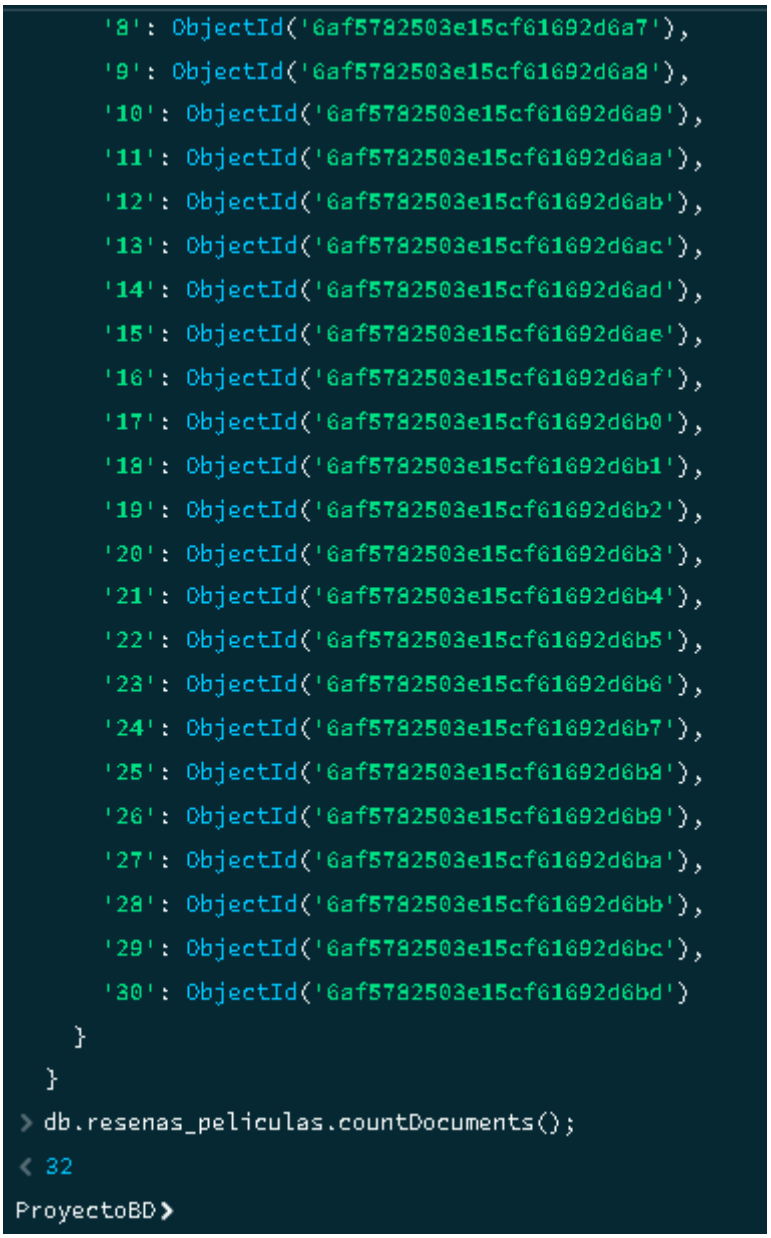


Fig. 6i. Carga de resenas_peliculas (insertMany, IDs insertados y countDocuments: 32).

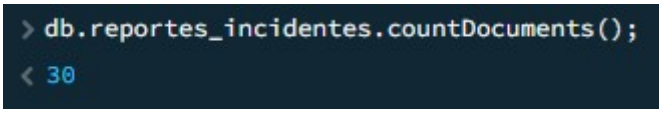


Fig. 6j. Conteo de documentos de reportes_incidentes (countDocuments: 30).

```
> db.reportes_incidentes.deleteMany({});

// Insercion valida que se ACEPTA (evidencia del $jsonSchema): falla tecnica de la funcion 4 (IMAX)
db.reportes_incidentes.insertOne({
  codigo: "INC-001",
  tipo: "falla_tecnica",
  fecha_hora: ISODate("2026-10-13T20:35:00-05:00"),
  id_funcion: 4,
  id_pelicula: 4,
  id_sala: 5,
  severidad: "alta",
  estado: "cerrado",
  descripcion: "Desalineación del segundo proyector IMAX durante el primer tercio de la película",
  reportado_por: {
    id_staff: 5,
    cargo: "Proyeccionista"
  },
  detalle_falla_tecnica: {
    equipo: "Proyector IMAX láser (unidad 2)",
    codigo_error: "ALN-204",
    minutos_interrupcion: 12,
    funcion_cancelada: false
  },
  acciones: [
    {
      fecha: ISODate("2026-10-13T20:37:00-05:00"),
      responsable_id_staff: 5,
      accion: "Pausa de la proyección y aviso al público"
    },
    {
      fecha: ISODate("2026-10-13T20:45:00-05:00"),
      responsable_id_staff: 5,
      accion: "Realineación manual del proyector"
    },
    {
      fecha: ISODate("2026-10-13T20:49:00-05:00"),
```

Fig. 6k. Inserción del incidente INC-001 en reportes_incidentes (insertOne).

```
< {
    acknowledged: true,
    insertedId: ObjectId('6af2baceadd2d0948f9b6744')
  }
```

Fig. 6l. Resultado de insertOne en reportes_incidentes (acknowledged: true).

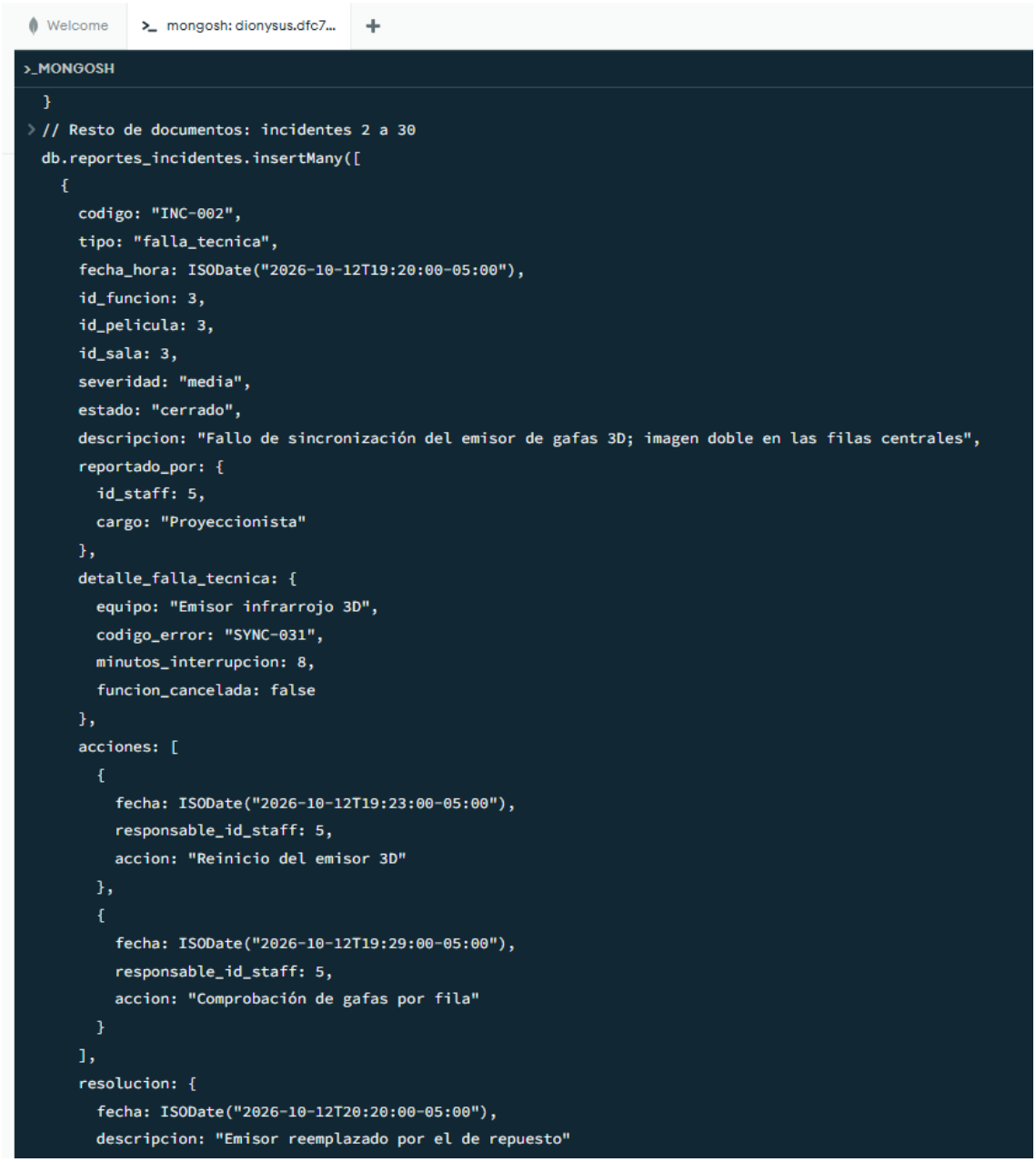


Fig. 6m. Carga de incidentes INC-002 a INC-030 en reportes_incidentes (insertMany).



Fig. 6n. Resultado de insertMany en reportes_incidentes (IDs insertados).

Colección	Índice	Tipo	Para qué sirve
`fichas_tecnicas_peliculas`	`{ id_pelicula: 1 }` (uq_ficha_id_pelicula)	Único	Una sola ficha por película; enlace con PostgreSQL.
`fichas_tecnicas_peliculas`	`{ "versiones.idioma_audio": 1 }` (ix_ficha_idioma_audio)	Multiclave	Buscar por idioma de audio de las versiones.
`requisitos_proyeccion`	`{ id_funcion: 1 }` (uq_id_funcion)	Único	Un documento de requisitos por función.
`requisitos_proyeccion`	`{ formato_sala: 1, id_funcion: 1 }` (ix_formato_funcion)	Compuesto	Funciones IMAX o 3D ordenadas por función.
`requisitos_proyeccion`	`{ id_pelicula: 1 }` (ix_id_pelicula)	Simple	Cruzar con fichas y reseñas.
`resenas_peliculas`	`{ id_resena: 1 }` (uq_id_resena)	Único	Identificador de la reseña.
`resenas_peliculas`	`{ id_pelicula: 1, fecha: -1 }` (ix_pelicula_fecha)	Compuesto	Reseñas más recientes de una película.
`resenas_peliculas`	`{ id_espectador: 1 }` (ix_espectador)	Simple	Reseñas de un espectador de PostgreSQL.
`resenas_peliculas`	`{ id_funcion: 1 }` (ix_funcion)	Parcial	Cruce por función, solo si la reseña tiene id_funcion.
`reportes_incidentes`	`{ codigo: 1 }` (uq_codigo)	Único	Código de incidente sin repetir.
`reportes_incidentes`	`{ tipo: 1, fecha_hora: -1 }` (ix_tipo_fecha)	Compuesto	Filtrar por tipo y ver lo más reciente.
`reportes_incidentes`	`{ id_funcion: 1 }` (ix_id_funcion)	Parcial	Cruce con requisitos_proyeccion, solo si hay función.
`reportes_incidentes`	`{ id_sala: 1, estado: 1 }` (ix_sala_estado)	Compuesto	Tablero de pendientes por sala.

Tabla 7. Los 13 índices definidos por colección (sin contar _id).

Consultas

Se ejecutaron seis consultas, una o más por colección, cada una por separado. Las consultas 3 y 6 usan $lookup entre colecciones de MongoDB, lo cual es posible porque ambas viven en el mismo motor.

La primera consulta es una agregación sobre fichas_tecnicas_peliculas, cuyo objetivo es identificar las películas con mayor número de idiomas y versiones, información que permite decidir cuáles requieren más copias y mayor programación (Fig. 7a a 7c). El pipeline desenrolla el arreglo versiones con $unwind, agrupa por película con $group, acumula los idiomas distintos mediante $addToSet y cuenta las versiones con $sum, calcula la cantidad de idiomas con $project y $size, y finalmente ordena y limita el resultado con $sort y $limit.

La segunda consulta es un find sobre requisitos_proyeccion que devuelve las funciones programadas en salas IMAX o 3D junto con su requisito técnico (Fig. 8a a 8c). El filtro se expresa con $in sobre el campo formato_sala, la proyección restringe los campos embebidos que necesita el equipo de proyección y el resultado se ordena por id_funcion.

La tercera consulta es una agregación sobre resenas_peliculas que calcula la calificación promedio de cada película, considerando únicamente las que tienen al menos dos reseñas (Fig. 9a a 9f). Con $group y $match se obtienen el promedio y el conteo de reseñas; con $lookup se enlaza cada resultado con fichas_tecnicas_peliculas para tomar el director, el género y la clasificación, y $unwind, $project y $sort dan la forma final. Junto con la consulta 6, es una de las dos que cruzan colecciones.

La cuarta consulta es un find sobre reportes_incidentes que recupera los incidentes de severidad alta o crítica que aún no están cerrados (Fig. 10a). Usa $in sobre los campos severidad y estado, proyecta los campos embebidos necesarios y ordena por fecha.

La quinta consulta es una agregación sobre reportes_incidentes que totaliza, para cada sala, los minutos perdidos y las funciones canceladas por fallas técnicas (Fig. 11a a 11c). Combina $match, $group, $sort y $project.

La sexta consulta es una agregación sobre reportes_incidentes que cruza las fallas técnicas con requisitos_proyeccion para contarlas por formato de sala. Usa $match, $lookup [5], $unwind, $group y $sort (Fig. 11d y 11e).

Los filtros y ordenamientos de estas consultas se apoyan en los índices descritos en la Tabla 7.

Resultados esperados.

- **Consulta 1:** 10 películas. Encabeza Avatar: El camino del agua (6 versiones, 2 idiomas), seguida de Dune: Parte Dos (5), Deadpool y Wolverine y Parásitos (4 cada una) y seis películas con 3 versiones.
- **Consulta 2:** 11 funciones (3, 4, 8, 9, 13, 14, 18, 21, 23, 26 y 27).

- **Consulta 3:** 15 películas. Cinema Paradiso y Coco encabezan con promedio 5,0 y Joker cierra con 3,0.
- **Consulta 4:** 2 incidentes, INC-009 (plataforma 4DX que no responde, sala 6) e INC-025 (hurto en el hall).
- **Consulta 5:** la sala 2 pierde más minutos (18 en 2 fallas y 1 función cancelada), seguida de la sala 5 (17), la sala 3 (8) y la sala 1 (6); las salas 4 y 6 tienen 1 falla sin minutos perdidos.
- **Consulta 6:** el formato 2D acumula más fallas (4, con 24 minutos perdidos), IMAX y 3D tienen 2 cada uno (17 y 8 minutos) y VIP 1. Con los datos de prueba, las fallas no se concentran en los formatos con más equipo.

```
>_MONGOSH
  { $unwind: "$versiones" },
  {
    $group: {
      _id: "$id_pelicula",
      titulo_pelicula: { $first: "$titulo_pelicula" },
      idiomas: { $addToSet: "$versiones.idioma_audio" },
      total_versiones: { $sum: 1 }
    }
  },
  {
    $project: {
      _id: 0,
      id_pelicula: "$_id",
      titulo_pelicula: 1,
      total_idiomas: { $size: "$idiomas" },
      total_versiones: 1,
      idiomas: 1
    }
  },
  { $sort: { total_versiones: -1, total_idiomas: -1, titulo_pelicula: 1 } },
  { $limit: 10 }
]);
```

Fig. 7a. Código de la consulta 1 en fichas_tecnicas_peliculas (pipeline de aggregate).

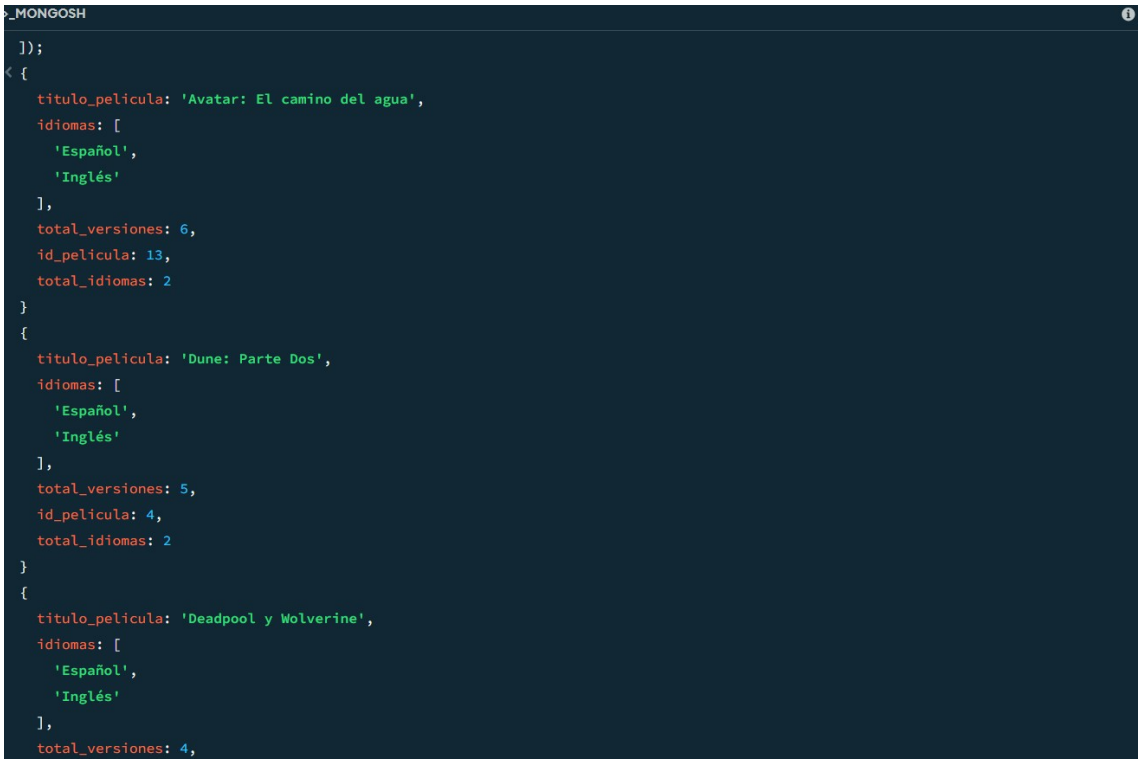


Fig. 7b. Resultados de la consulta 1 (fragmento 1).

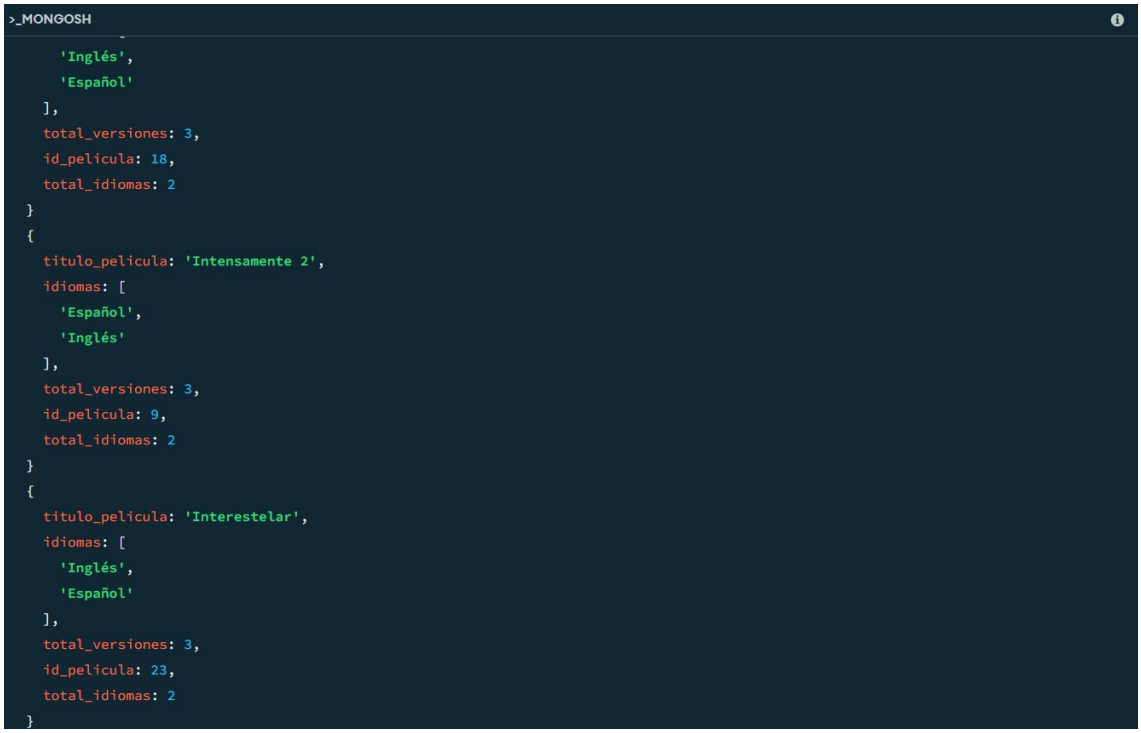


Fig. 7c. Resultados de la consulta 1 (fragmento 2).

```
> db.requisitos_proyeccion.find(
    { formato_sala: { $in: ["IMAX", "3D"] } },
    {
      _id: 0,
      id_funcion: 1,
      titulo_pelicula: 1,
      formato_sala: 1,
      "proyeccion.resolucion": 1,
      "proyeccion.relacion_aspecto": 1,
      "proyeccion.fps": 1,
      "audio.formato": 1,
      "requisitos_3d.tipo_gafas": 1,
      "requisitos_3d.gafas_requeridas": 1,
      "requisitos_imax.alineacion_doble_proyector": 1,
      "ensayo_tecnico.fecha": 1
    }
  ).sort({ id_funcion: 1 });
< {
    id_funcion: 3,
    titulo_pelicula: 'Parásitos',
    formato_sala: '3D',
    proyeccion: {
      resolucion: '4K',
```

Fig. 8a. Consulta 2 en requisitos_proyeccion (find con $in y proyección).



Fig. 8b. Resultados de la consulta 2 (funciones IMAX y 3D).

```
>_MONGOSH
// Proposito: que el equipo de proyeccion sepa que preparar para las
// funciones que exigen equipo especial (gafas 3D, doble proyector IMAX,
// relacion de aspecto, ensayo tecnico).
// Tecnicas: $in sobre formato_sala, proyeccion de campos embebidos,
// orden por id_funcion. Usa el indice ix_formato_funcion.

db.requisitos_proyeccion.find(
  { formato_sala: { $in: ["IMAX", "3D"] } },
  {
    _id: 0,
    id_funcion: 1,
    titulo_pelicula: 1,
    formato_sala: 1,
    "proyeccion.resolucion": 1,
    "proyeccion.relacion_aspecto": 1,
    "proyeccion.fps": 1,
    "audio.formato": 1,
    "requisitos_3d.tipo_gafas": 1,
    "requisitos_3d.gafas_requeridas": 1,
    "requisitos_imax.alineacion_doble_proyector": 1,
    "ensayo_tecnico.fecha": 1
  }
).sort({ id_funcion: 1 });
```

Fig. 8c. Código comentado de la consulta 2 en requisitos_proyeccion.



```
    { $sort: { calificacion_promedio: -1, total_resenas: -1, titulo: 1 } }
  ]);

< {
    total_resenas: 2,
    resenas_respondidas: 1,
    id_pelicula: 11,
    titulo: 'Cinema Paradiso',
    director: 'Giuseppe Tornatore',
    generos: [
      'Drama'
    ],
    clasificacion: '+7',
    calificacion_promedio: 5
  }
  {
    total_resenas: 2,
    resenas_respondidas: 0,
    id_pelicula: 2,
    titulo: 'Coco',
    director: 'Lee Unkrich',
    generos: [
      'Animación',
      'Familiar'
    ],
    clasificacion: 'Todo publico',
    calificacion_promedio: 5
  }
```

Fig. 9a. Resultados de la consulta 3 (fragmento 1 de 6).

```
{
  total_resenas: 2,
  resenas_respondidas: 0,
  id_pelicula: 13,
  titulo: 'Avatar: El camino del agua',
  director: 'James Cameron',
  generos: [
    'Ciencia ficción',
    'Aventura'
  ],
  clasificacion: '+12',
  calificacion_promedio: 4.5
}
{
  total_resenas: 2,
  resenas_respondidas: 0,
  id_pelicula: 14,
  titulo: 'El Origen',
  director: 'Christopher Nolan',
  generos: [
    'Ciencia ficción',
    'Acción'
  ],
  clasificacion: '+12',
  calificacion_promedio: 4.5
}
```


Fig. 9b. Resultados de la consulta 3 (fragmento 2 de 6).

```
  total_resenas: 2,
  resenas_respondidas: 0,
  id_pelicula: 9,
  titulo: 'Intensamente 2',
  director: 'Kelsey Mann',
  generos: [
    'Animación',
    'Comedia'
  ],
  clasificacion: 'Todo publico',
  calificacion_promedio: 4.5
}
{
  total_resenas: 2,
  resenas_respondidas: 0,
  id_pelicula: 5,
  titulo: 'La La Land',
  director: 'Damien Chazelle',
  generos: [
    'Musical',
    'Romance'
  ],
  clasificacion: '+7',
  calificacion_promedio: 4.5
}
```


Fig. 9c. Resultados de la consulta 3 (fragmento 3 de 6).

```
  resenas_respondidas: 0,
  id_pelicula: 12,
  titulo: 'Mad Max: Furia en el camino',
  director: 'George Miller',
  generos: [
    'Acción',
    'Ciencia ficción'
  ],
  clasificacion: '+15',
  calificacion_promedio: 4.5
}
{
  total_resenas: 2,
  resenas_respondidas: 1,
  id_pelicula: 1,
  titulo: 'Oppenheimer',
  director: 'Christopher Nolan',
  generos: [
    'Drama',
    'Biografía'
  ],
  clasificacion: '+15',
  calificacion_promedio: 4.5
}
{
  total_resenas: 2,
  resenas_respondidas: 0,
  id_pelicula: 3,
```


Fig. 9d. Resultados de la consulta 3 (fragmento 4 de 6).

```
  titulo: 'Parásitos',
  director: 'Bong Joon-ho',
  generos: [
    'Thriller',
    'Drama'
  ],
  clasificacion: '+15',
  calificacion_promedio: 4.5
}
{
  total_resenas: 2,
  resenas_respondidas: 0,
  id_pelicula: 15,
  titulo: 'Spider-Man: Un nuevo universo',
  director: 'Bob Persichetti',
  generos: [
    'Animación',
    'Acción'
  ],
  clasificacion: '+7',
  calificacion_promedio: 4.5
}
{
  total_resenas: 3,
  resenas_respondidas: 0,
  id_pelicula: 10,
  titulo: 'Barbie',
  director: 'Greta Gerwig',
```


Fig. 9e. Resultados de la consulta 3 (fragmento 5 de 6).



Fig. 9f. Resultados de la consulta 3 (fragmento 6 de 6).

```
> db.reportes_incidentes.find(
    { severidad: { $in: ["alta", "critica"] }, estado: { $in: ["abierto", "en_revision"] } },
    { _id: 0, codigo: 1, tipo: 1, fecha_hora: 1, id_sala: 1, id_funcion: 1, severidad: 1, estado: 1, descripcion: 1, "repor
  ).sort({ fecha_hora: -1 });
< {
    codigo: 'INC-009',
    tipo: 'falla_tecnica',
    fecha_hora: ISODate('2026-11-02T15:30:00.000Z'),
    id_sala: 6,
    severidad: 'alta',
    estado: 'en_revision',
    descripcion: 'La plataforma de movimiento 4DX no responde durante la revisión de mantenimiento',
    reportado_por: {
      cargo: 'Coordinador'
    }
  }
  {
    codigo: 'INC-025',
    tipo: 'incidente_seguridad',
    fecha_hora: ISODate('2026-10-25T01:10:00.000Z'),
    severidad: 'alta',
    estado: 'en_revision',
    descripcion: 'Hurto de un bolso en el hall durante la hora pico',
    reportado_por: {
      cargo: 'Administrador'
    }
  }
```

Fig. 10a. Consulta 4 en reportes_incidentes (find de incidentes graves abiertos y resultado).

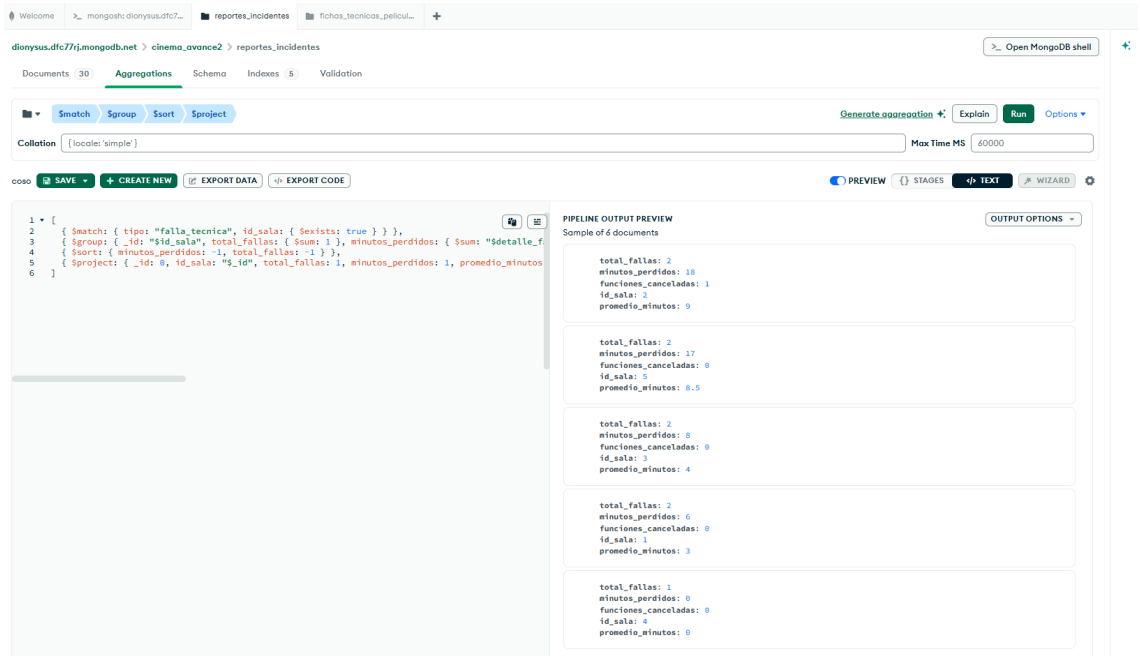


Fig. 11a. Consulta 5 en reportes_incidentes en Compass (aggregate).

```
> db.reportes_incidentes.aggregate([
    { $match: { tipo: "falla_tecnica", id_sala: { $exists: true } } },
    {
      $group: {
        _id: "$id_sala",
        total_fallas: { $sum: 1 },
        minutos_perdidos: { $sum: "$detalle_falla_tecnica.minutos_interrupcion" },
        promedio_minutos: { $avg: "$detalle_falla_tecnica.minutos_interrupcion" },
        funciones_canceladas: {
          $sum: { $cond: ["$detalle_falla_tecnica.funcion_cancelada", 1, 0] }
        }
      }
    },
    { $sort: { minutos_perdidos: -1, total_fallas: -1 } },
    {
      $project: {
        _id: 0,
        id_sala: "$_id",
        total_fallas: 1,
        minutos_perdidos: 1,
        promedio_minutos: { $round: ["$promedio_minutos", 1] },
        funciones_canceladas: 1
      }
    }
  ]);
< {
    total_fallas: 2,
    minutos_perdidos: 18,
    funciones_canceladas: 1,
    id_sala: 2,
    promedio_minutos: 9
  }
  {
    total_fallas: 2,
```

Fig. 11b. Código y resultado de la consulta 5 (aggregate por sala).



Fig. 11c. Resultados de la consulta 5 (fragmento).

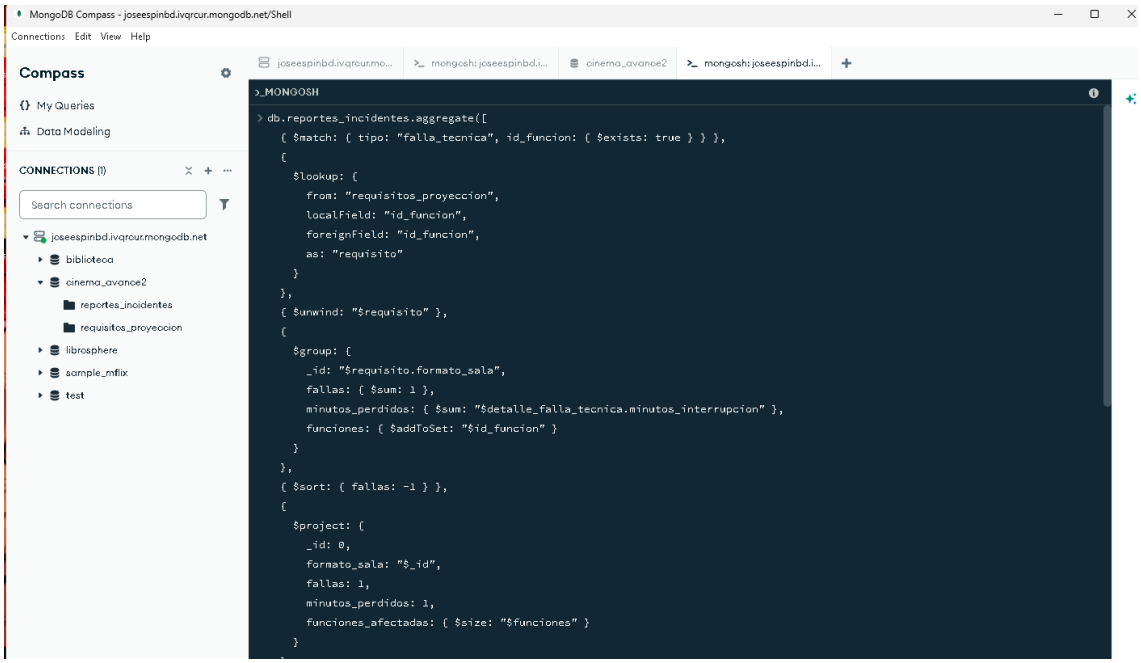


Fig. 11d. Código de la consulta 6 en reportes_incidentes (aggregate con $lookup).

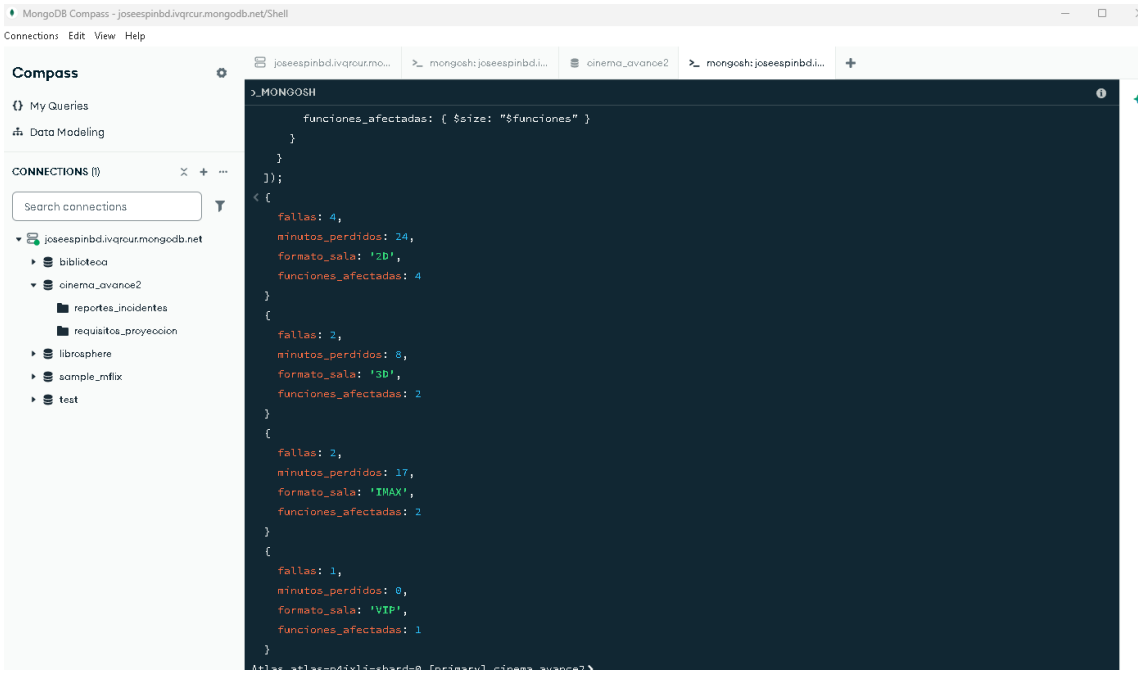


Fig. 11e. Resultado de la consulta 6 (fallas técnicas por formato de sala).

Validación: inserciones rechazadas

Cada script de consultas termina con una inserción inválida que MongoDB debe rechazar con «Document failed validation»: una ficha sin `id_pelicula`, `ficha` ni `actualizado_en` y con versiones vacías; un requisito sin `id_funcion` y con formato «8K»; una reseña sin `id_espectador`, con calificación 7 y comentario de dos letras; y una falla técnica sin

`detalle_falla_tecnica` y con severidad «gravisima». La primera inserción de cada script 02 (ficha 1, función 1, reseña 1 e INC-001) es la contraparte válida y debe aceptarse.

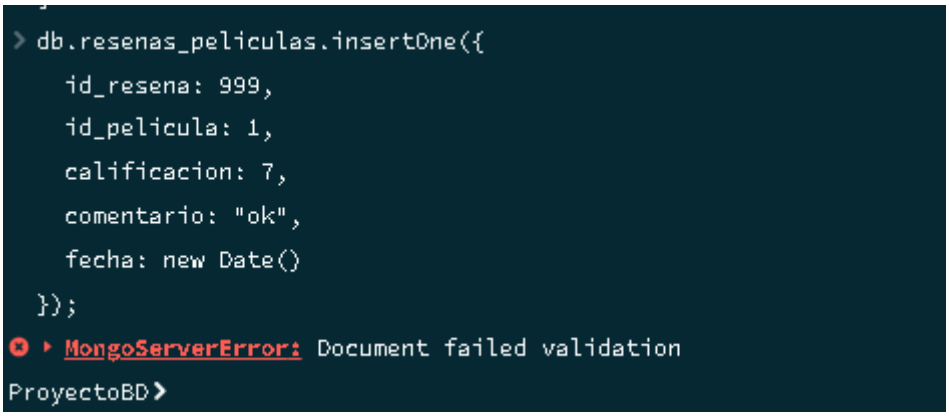


Fig. 12a. Inserción rechazada en resenas_peliculas (Document failed validation).

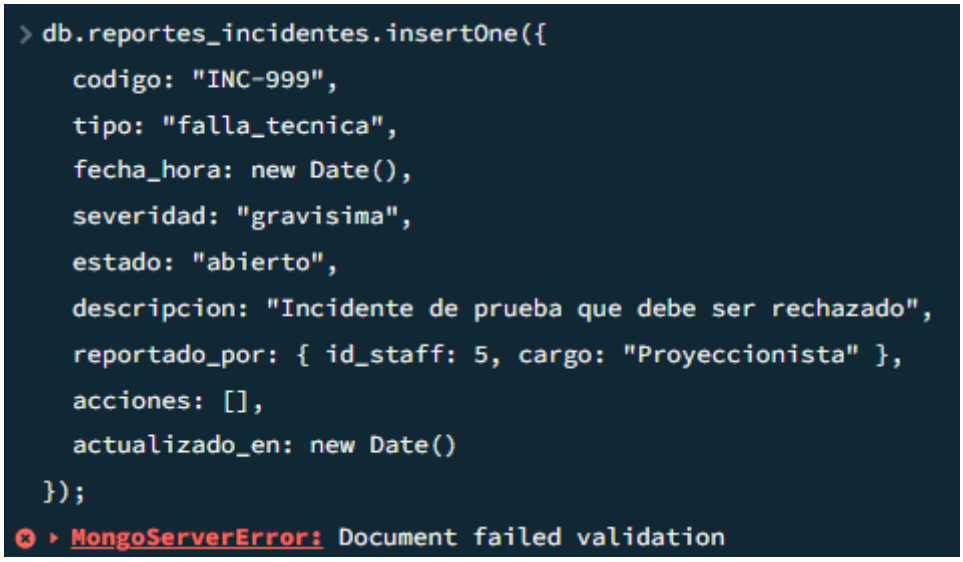


Fig. 12b. Inserción rechazada en reportes_incidentes (Document failed validation).

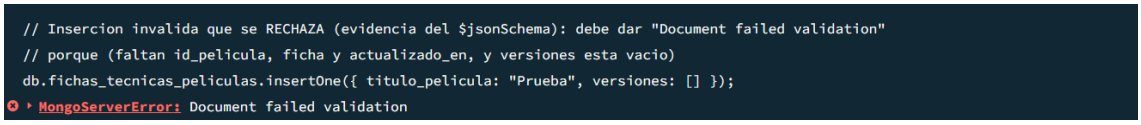


Fig. 12c. Inserción rechazada en fichas_tecnicas_peliculas (Document failed validation).

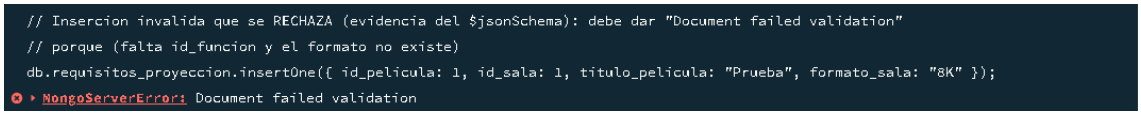


Fig. 12d. Inserción rechazada en requisitos_proyeccion (Document failed validation).

Justificación teórica: ACID, BASE y CAP

La decisión de qué dato va en cada motor se apoya en dos modelos de garantías, ACID y BASE, y en el teorema CAP, que explica por qué no se pueden tener todas las garantías a la vez cuando un sistema es distribuido.

ACID en el núcleo transaccional

Una transacción ACID cumple cuatro propiedades [6], [7], y cada una tiene un ejemplo concreto en el proyecto:

- **Atomicidad.** Una transacción se ejecuta completa o no se ejecuta. `sp_cancelar_funcion` anula las ventas, cancela los contratos y marca la función como cancelada en una sola transacción; si algo falla a la mitad, todo se revierte y no queda una función cancelada con ventas pagadas.
- **Consistencia.** La base pasa siempre de un estado válido a otro válido. Las llaves foráneas, los CHECK, los UNIQUE y los ocho triggers impiden estados como vender más boletas que la capacidad de la sala, programar dos funciones que se cruzan o contratar a un artista inactivo.
- **Aislamiento.** Las transacciones concurrentes no se interfieren. El trigger de cupo bloquea la fila de la función con `FOR UPDATE`: si dos clientes compran la última silla al mismo tiempo, la segunda venta espera, ve la primera y es rechazada, en lugar de vender dos veces la misma silla.
- **Durabilidad.** Una vez confirmada, una venta sobrevive a una caída del servidor porque PostgreSQL registra los cambios en el registro de escritura anticipada (WAL) antes de confirmar.

Por eso ventas, contratos, puntos de fidelización, capacidad y estado de las funciones permanecen en PostgreSQL: son datos donde un error se traduce directamente en dinero o en una promesa incumplida al cliente.

BASE en los datos documentales

BASE describe sistemas que priorizan la disponibilidad y la escala sobre la consistencia inmediata: *Basically Available* (el sistema responde aunque una parte falle), *Soft state* (el estado puede cambiar sin nuevas entradas mientras los datos se propagan) y *Eventually consistent* (si no hay nuevas escrituras, las réplicas terminan coincidiendo) [8]–[10].

En Altamira Cine esto es aceptable para las cuatro colecciones: la calificación promedio de una película, calculada desde un nodo secundario, puede no incluir todavía la reseña más reciente (estado «blando»); una ficha técnica leída desde un secundario puede no reflejar el último cambio (consistencia eventual); y la plataforma sigue mostrando fichas y reseñas aunque el servicio de ventas esté ocupado (disponibilidad). Ninguna de esas demoras causa un perjuicio económico.

MongoDB sí ofrece transacciones multidocumento desde la versión 4.0, pero el diseño no las necesita: lo que debe cambiar junto se embebió en un mismo documento (las acciones de seguimiento de un incidente, la respuesta de la empresa a una reseña) y una escritura sobre un documento es atómica.

Aspecto	ACID (PostgreSQL)	BASE (MongoDB)
Prioridad	Corrección de cada transacción	Disponibilidad y escalabilidad
Consistencia	Inmediata y obligatoria (restricciones, triggers)	Eventual; configurable por operación
Esquema	Fijo, definido de antemano (DDL)	Flexible, con validación ($jsonSchema)
Relaciones	Llaves foráneas y JOINs	Embebido y referencias por id; sin FK
Concurrencia	Bloqueos de fila (FOR UPDATE) y aislamiento	Atomicidad por documento; menos bloqueos
Uso en Altamira Cine	Ventas, contratos, funciones, puntos	Fichas, requisitos, reseñas, incidentes

Tabla 8. Comparación entre el modelo ACID y el modelo BASE aplicada al proyecto.

Teorema CAP

El teorema CAP (Brewer) establece que un sistema distribuido solo puede garantizar dos de tres propiedades: Consistencia (todos los nodos ven el mismo dato más reciente), Disponibilidad (toda solicitud recibe respuesta) y Tolerancia a particiones (el sistema sigue funcionando aunque se pierda la comunicación entre nodos). Como las particiones de red ocurren en la práctica, la elección real es entre consistencia y disponibilidad mientras dura una partición [11]–[13]. La Fig. 13 ubica los dos motores del proyecto según ese criterio.

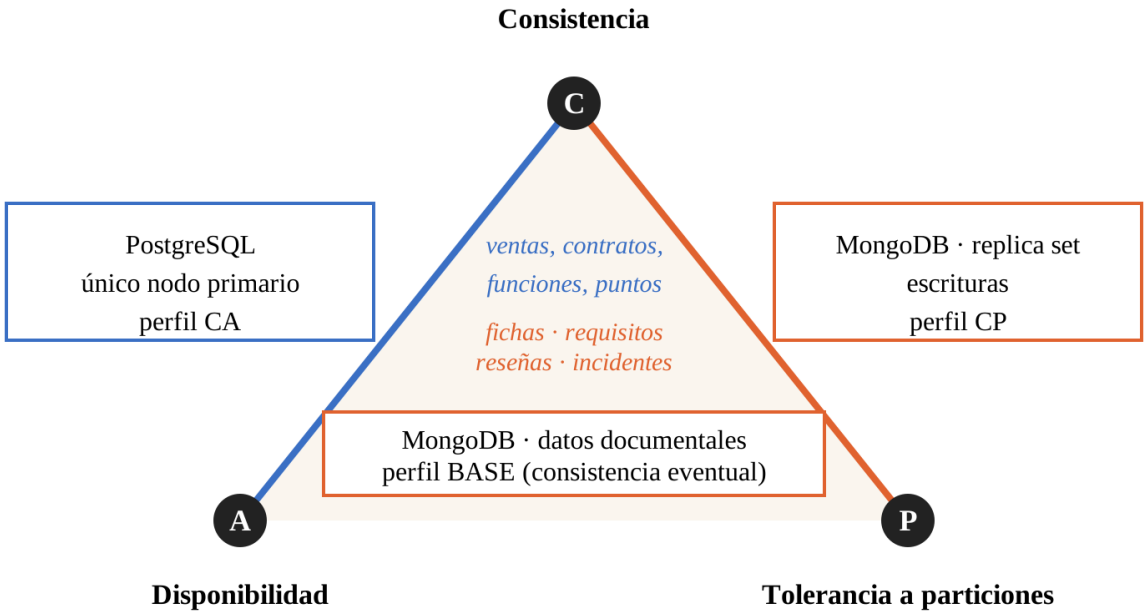


Fig. 13. Teorema CAP y ubicación de los dos motores del proyecto.

PostgreSQL se utiliza como un único nodo primario. Con un solo nodo no hay particiones que tratar, por lo que ofrece consistencia fuerte y disponibilidad (perfil CA) mientras el nodo esté operativo. Si en un despliegue futuro se añadieran réplicas síncronas [16], el sistema tendría que rechazar las escrituras que no pueda confirmar en todas, es decir, preferiría la consistencia (perfil CP). Esa es la elección correcta para ventas: es preferible rechazar una compra a vender una silla que no existe.

MongoDB se despliega como replica set con un nodo primario que recibe las escrituras. Si una partición aísla al primario de la mayoría de los nodos, las escrituras que requieren confirmación de mayoría no se completan hasta que se restablezca el quórum, lo que corresponde a un comportamiento CP. Por defecto MongoDB se comporta como CP. Sin embargo, para los datos documentales del proyecto se aceptan lecturas desde nodos secundarios, que pueden estar levemente atrasados [14], [15], y por eso el criterio de uso es BASE: priorizar la disponibilidad y la escala, aceptando que las réplicas converjan con el tiempo.

Conclusión

El Avance 2 completa el sistema de Altamira Cine con una arquitectura políglota. El núcleo transaccional en PostgreSQL, corregido según la retroalimentación del Avance 1 (nueva entidad *peliculas*, modelo relacional alineado con el script, tercera forma normal sin excepciones, evidencias con sentencia y resultado), mantiene las nueve tablas, los ocho triggers, los tres procedimientos y las tres consultas. Sobre esa base, MongoDB aporta cuatro colecciones (*fichas_tecnicas_peliculas*, *requisitos_proyeccion*, *resenas_peliculas* y *reportes_incidentes*) diseñadas con criterios explícitos de embebido y referencia, validación con $jsonSchema, 13 índices propios y seis consultas (dos con find y cuatro con *aggregation*, entre ellas $unwind y $lookup).

La justificación ACID, BASE y CAP muestra por qué cada dato está donde está: consistencia fuerte para lo transaccional y flexibilidad, volumen y disponibilidad para lo documental. Como trabajo posterior queda evaluar colecciones adicionales, como los eventos de la plataforma con un índice TTL.

Referencias

[1] M. Fowler, “Polyglot persistence,” martinfowler.com, 2011. [En línea]. Disponible en: https://martinfowler.com/bliki/PolyglotPersistence.html [Consultado: 12-nov-2026].

[2] Escuela Politécnica Nacional, Repositorio Digital EPN, ítem 15000/27618. [En línea]. Disponible en: https://bibdigital.epn.edu.ec/handle/15000/27618 [Consultado: 12-nov-2026].

[3] MongoDB, Inc., “Data Modeling,” MongoDB Manual. [En línea]. Disponible en: https://www.mongodb.com/docs/manual/data-modeling/ [Consultado: 12-nov-2026].

[4] MongoDB, Inc., “Schema Validation,” MongoDB Manual. [En línea]. Disponible en: https://www.mongodb.com/docs/manual/core/schema-validation/ [Consultado: 12-nov-2026].

[5] MongoDB, Inc., “$lookup (aggregation),” MongoDB Manual. [En línea]. Disponible en: https://www.mongodb.com/docs/manual/reference/operator/aggregation/lookup/ [Consultado: 12-nov-2026].

[6] T. Härder and A. Reuter, “Principles of transaction-oriented database recovery,” ACM Computing Surveys, vol. 15, no. 4, pp. 287–317, Dec. 1983, doi: 10.1145/289.291.

[7] PostgreSQL Global Development Group, “3.4. Transactions,” PostgreSQL 17 Documentation. [En línea]. Disponible en: https://www.postgresql.org/docs/17/tutorial-transactions.html [Consultado: 12-nov-2026].

[8] D. Pritchett, “BASE: An ACID alternative,” ACM Queue, vol. 6, no. 3, pp. 48-55, May/Jun. 2008. [En línea]. Disponible en: https://queue.acm.org/detail.cfm?id=1394128

[9] W. Vogels, “Eventually consistent,” Commun. ACM, vol. 52, no. 1, pp. 40-44, Jan. 2009. [En línea]. Disponible en: https://cs.brown.edu/courses/csci2950-u/f09/papers/p40-vogels.pdf

[10] Cornell Univ., “BASE versus ACID,” CS5412 lecture slides, 2014. [En línea]. Disponible en: https://www.cs.cornell.edu/Courses/cs5412/2014sp/slides/IX%20-%20BASE%20versus%20ACID.pdf

[11] S. Gilbert and N. Lynch, “Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services,” ACM SIGACT News, vol. 33, no. 2, pp. 51–59, Jun. 2002, doi: 10.1145/564585.564601. [En línea]. Disponible en: https://www.comp.nus.edu.sg/~gilbert/pubs/BrewersConjecture-SigAct.pdf

[12] E. Brewer, “A certain freedom: Thoughts on the CAP theorem,” in Proc. 29th ACM SIGACT-SIGOPS Symp. Principles of Distributed Computing (PODC ’10), 2010, p. 335, doi: 10.1145/1835698.1835701. [En línea]. Disponible en: https://dl.acm.org/doi/abs/10.1145/1835698.1835701

[13] “The CAP FAQ,” The Paper Trail. [En línea]. Disponible en: https://www.the-paper-trail.org/page/cap-faq/ [Consultado: 12-nov-2026].

[14] MongoDB, Inc., “Read Preference,” MongoDB Manual. [En línea]. Disponible en: https://www.mongodb.com/docs/manual/core/read-preference/ [Consultado: 12-nov-2026].

[15] MongoDB, Inc., “Read Preference Use Cases,” MongoDB Manual. [En línea]. Disponible en: https://www.mongodb.com/docs/manual/core/read-preference-use-cases/ [Consultado: 12-nov-2026].

[16] PostgreSQL Global Development Group, “19.6. Replication,” PostgreSQL 17 Documentation. [En línea]. Disponible en: https://www.postgresql.org/docs/17/runtime-config-replication.html [Consultado: 12-nov-2026].